Stream optimization algorithm in data transmission networks

Developed by: *Vladimir Storozhuk*
Email: jarilydar@gmail.com
Tel.: +7 999 442 08 91

Content list

Preamble……..3

1. Mathematical optimization algorithm……..……4

1.1 Formulating of the optimization problem……4

1.2 Data network structure……...5

1.3 Subflows incidence equations ……...7

1.4 Objective function……10

2. Optimization calculation example…..…...……13

2.1 Initial data ……....13

2.2 Objective function limitations…..……14

2.3 Objective function ……18

2.4 The matrixes of the optimal subflows….……..20

2.5 Optimal subflows visualization……..……...22

3. Optimization algorithm program code…….……25

3.1 OptiFlows.m script…….……....25

3.2 TargetFunction.m function..……27

4. Program code line-by-line comments..……...28

4.1 OptiFlows.m script……....28

4.2 TargetFunction.m function..……35

Epilogue…….....36

Preamble

The efficiency of business processes of companies directly depends on the performance of corporate infocommunication systems, and the performance of modern infocommunication systems can be significantly increased not only by expensive network scaling and increasing the bandwidth of communication channels, but also by optimizing the distribution of information flows, which is practically insignificant in terms of costs.

This article presents a software of mathematical algorithm for automatic optimization of information flows in data transmission networks.

The scientific novelty of the presented algorithm lies in the absence at the time of publication of this work of existing similar software-implemented algorithms, at least in the public domain.

The economic effect of using this algorithm will be a significant reduction in the cost of maintenance and modernization of data transmission networks.

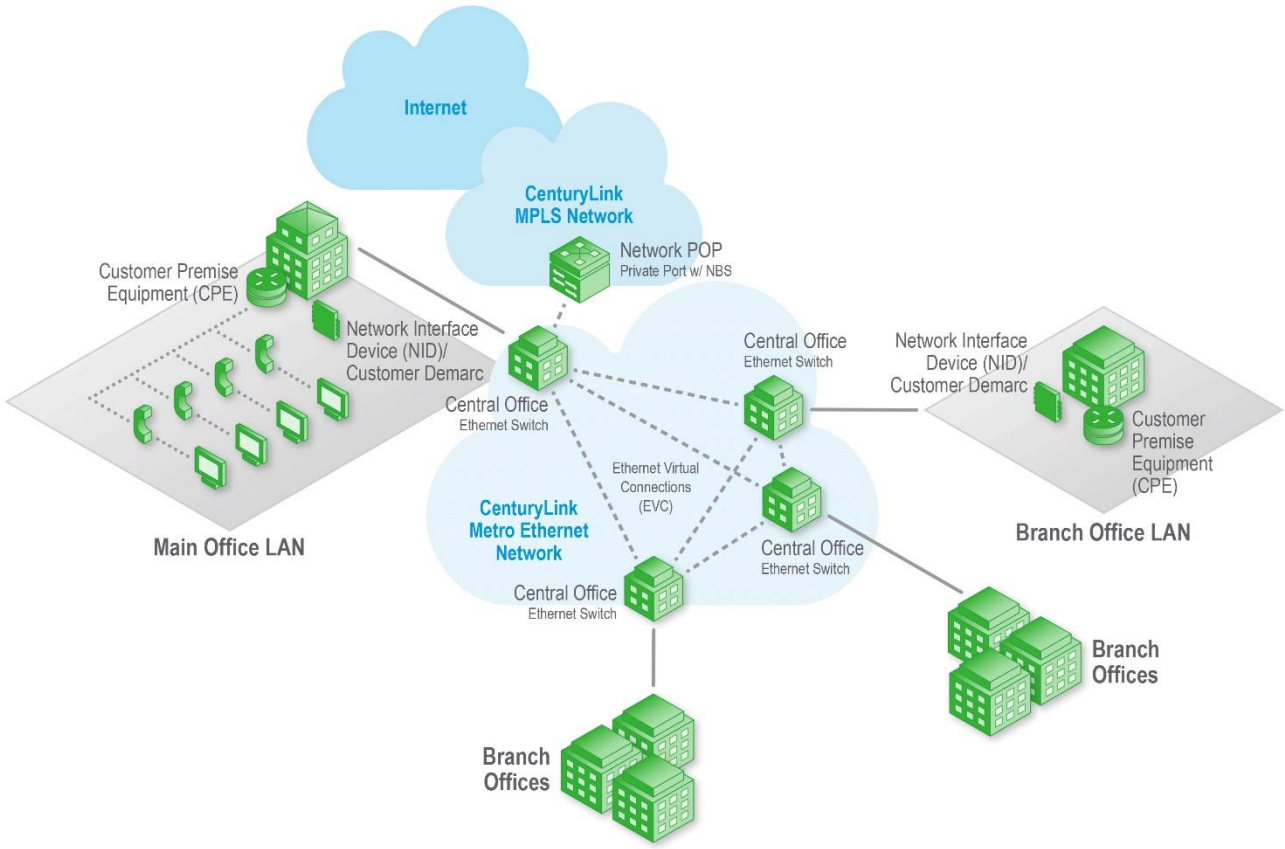

Figure 1 – Example of a network

1. Mathematical optimization algorithm

1.1 Formulating of the optimization problem

According to the Queuing theory, the average queue of service requests is determined by the formula:

$$N = \frac{\frac{\lambda}{\mu}}{1 - \frac{\lambda}{\mu}},$$

where $\boldsymbol{\lambda}$ is service requests intensity, $\boldsymbol{\mu}$ – channel bandwidth.

In the modern communication systems, service requests intensity can be represented like the value of information flows, and can be measured as well as channel bandwidth, in megabits per second for example.

Depending on the structure of the queuing system, there may be different amounts of information flows, channel bandwidth, and combinations thereof.

The task of optimizing the distribution of information flows is reduced to finding such values of $\boldsymbol{\lambda}$, so that the value of the objective function $\boldsymbol{N}$ is minimal, since with the minimum value of the average queue of service requests, the data transmission network will have maximum performance.

1.2 Data network structure

The structure of the data transmission network can be visualized using a graph:

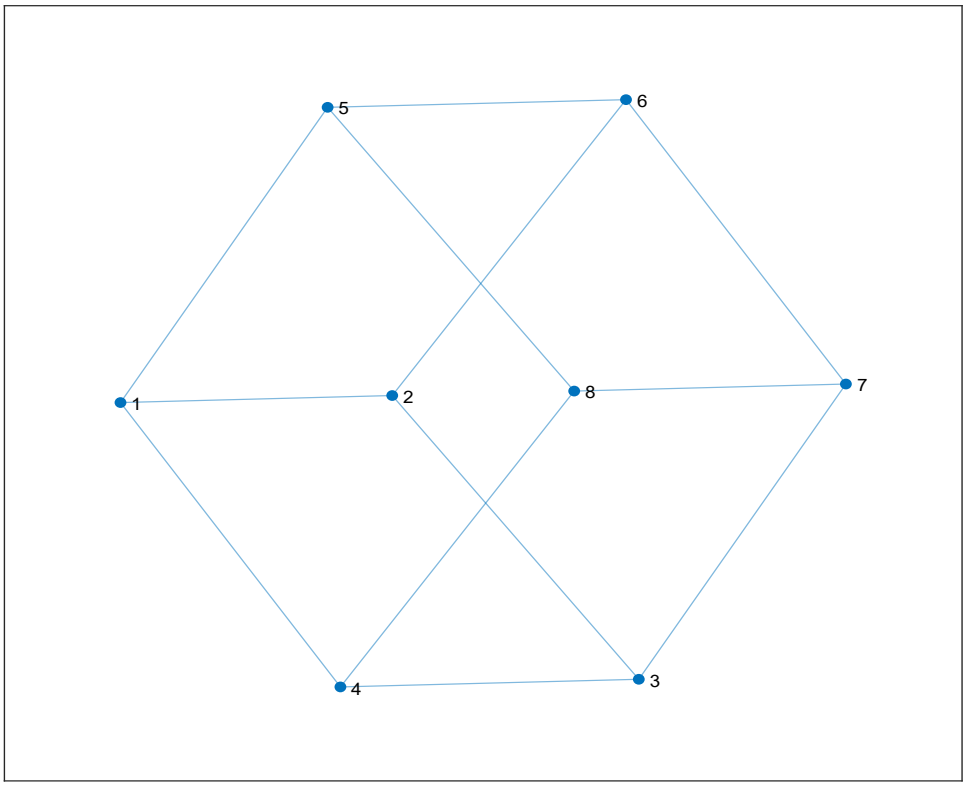

Figure 1.2.1 – Undirected graph

The graph shown in Figure 1.2.1 has 8 vertices (nodes) and 12 edges (branches) and is non-directional.

However, most data transmission networks have both forward and reverse directions of communication channels, so it is better to visualize the network most accurately using a directed graph:

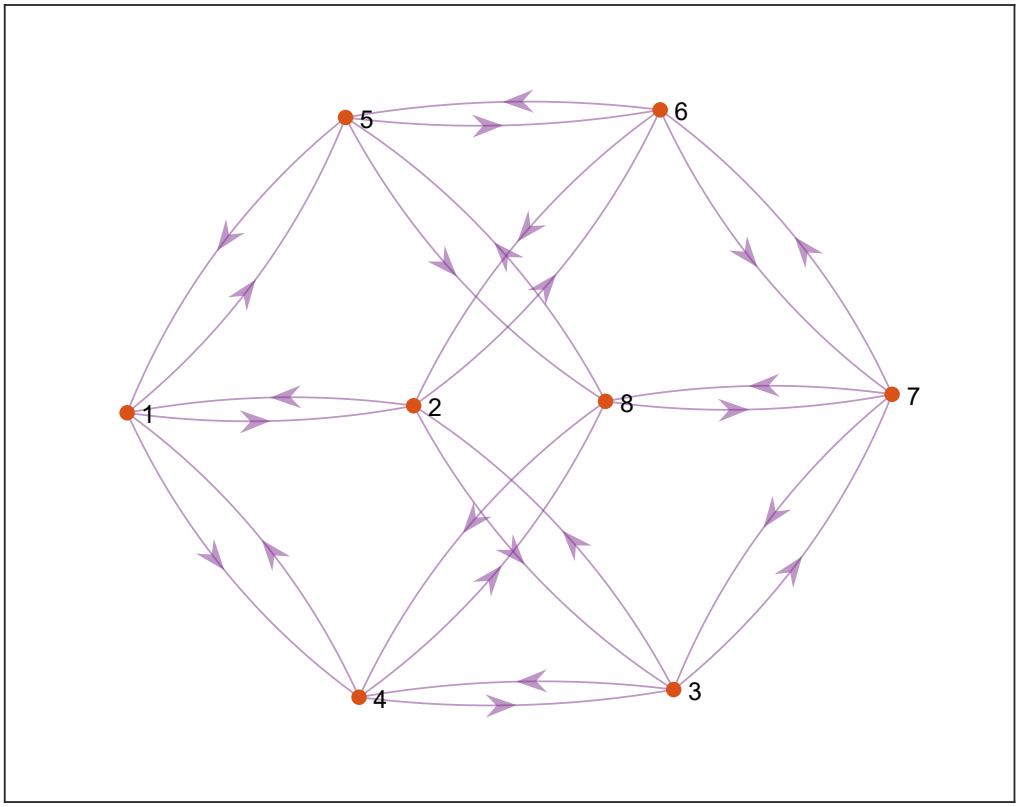

Figure 1.2.2 – Directed graph

Figure 1.2.2 shows a directed graph consisting of 8 nodes and 24 edges indicating the directions of communication channels.

The structure of any data transmission network can be represented as a square matrix of a network with a zero diagonal of size $\boldsymbol{n}\times\boldsymbol{n}$:

$$M = \begin{matrix} 0 & \mu_{1,2} & \mu_{1,3} & \cdots & \mu_{1,n} \\ \mu_{2,1} & 0 & \mu_{2,3} & \cdots & \mu_{2,n} \\ \mu_{3,1} & \mu_{3,2} & 0 & \cdots & \mu_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mu_{n,1} & \mu_{n,2} & \mu_{n,3} & \cdots & 0 \end{matrix}$$

, where n is the number of nodes in the network, $\mu_{i,j}$ – is the total bandwidth of communication channels (graph branches) connecting and directed from node number i to node number j, and the zero diagonal implies the absence of channels for information flows from the node addressed to itself.

Similarly, the direction of flows in any data transmission network can also be represented as a square matrix of flows with a zero diagonal of size $\boldsymbol{n}\times\boldsymbol{n}$:

$$F = \begin{matrix} 0 & \varphi_{1,2} & \varphi_{1,3} & \cdots & \varphi_{1,n} \\ \varphi_{2,1} & 0 & \varphi_{2,3} & \cdots & \varphi_{2,n} \\ \varphi_{3,1} & \varphi_{3,2} & 0 & \cdots & \varphi_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \varphi_{n,1} & \varphi_{n,2} & \varphi_{n,3} & \cdots & 0 \end{matrix}$$

, where $\boldsymbol{n}$ is the number of nodes in the network, $\boldsymbol{\varphi_{i,j}}$ – the value of the information stream, directed from node number $\boldsymbol{i}$ to node number $\boldsymbol{j}$. The zero diagonal in the matrix implies the absence of information streams from the node addressed to itself.

The flow directions can be represented as a string:

$$\varphi_1 \quad \varphi_2 \quad \varphi_3 \quad \cdots \quad \varphi_m,$$

where $\boldsymbol{m}$ is the number of information streams, $\boldsymbol{\varphi_k}$ is a non-zero information flow number $\boldsymbol{k}$. Flow number $\boldsymbol{k}$ assigned based on the matrix of flows in order line by line.

We take the matrix of the network $\boldsymbol{M}$ and the matrix of flows $\boldsymbol{F}$ as the initial data for the optimization problem.

In the process of data transmission in a network of a certain structure, after the outgoing channel, the stream is divided into certain fractions of streams (sub-streams) passing through other channels, and then the sub-streams are combined again into a single stream in the incoming channel.

1.3 Sub-streams incidence equations

The structure of sub-streams can be represented as a system of matrices in the amount of ***m*** of size ***n*×*n*** :

$$\left\{\begin{array}{c}
\begin{matrix} 0 & \lambda_{1,2,1} & \lambda_{1,3,1} & \cdots & \lambda_{1,n,1} \\ \lambda_{2,1,1} & 0 & \lambda_{2,3,1} & \cdots & \lambda_{2,n,1} \\ \lambda_{3,1,1} & \lambda_{3,2,1} & 0 & \cdots & \lambda_{3,n,1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,1} & \lambda_{n,2,1} & \lambda_{n,3,1} & \cdots & 0 \end{matrix} ; \\
\begin{matrix} 0 & \lambda_{1,2,2} & \lambda_{1,3,2} & \cdots & \lambda_{1,n,2} \\ \lambda_{2,1,2} & 0 & \lambda_{2,3,2} & \cdots & \lambda_{2,n,2} \\ \lambda_{3,1,2} & \lambda_{3,2,2} & 0 & \cdots & \lambda_{3,n,2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,2} & \lambda_{n,2,2} & \lambda_{n,3,2} & \cdots & 0 \end{matrix} ; \\
\vdots \\
\begin{matrix} 0 & \lambda_{1,2,m} & \lambda_{1,3,m} & \cdots & \lambda_{1,n,m} \\ \lambda_{2,1,m} & 0 & \lambda_{2,3,m} & \cdots & \lambda_{2,n,m} \\ \lambda_{3,1,m} & \lambda_{3,2,m} & 0 & \cdots & \lambda_{3,n,m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,m} & \lambda_{n,2,m} & \lambda_{n,3,m} & \cdots & 0 \end{matrix} .
\end{array}\right\}$$

, where ***n*** is the number of nodes in the network, ***m*** is the number of information streams, $\boldsymbol{\lambda_{i,j,k}}$ is the fraction of the flow number ***k***, passing through the channel directed from node number ***i*** к to node number ***j***.

Based on the matrices of stream values and stream directions, we will compile a system of incidence equations for sub-streams directed from ***node 1*** to ***node 2*** ($\boldsymbol{\varphi_{1,2}}$ ***stream***):

$$\left\{\begin{array}{l}
\text{Node 1: } -\lambda_{1,2,1} - \lambda_{1,3,1} - \cdots - \lambda_{1,n,1} + \lambda_{2,1,1} + \lambda_{3,1,1} + \cdots + \lambda_{n,1,1} = 0 \\
\text{Node 2: } -\lambda_{2,1,1} - \lambda_{2,3,1} - \cdots - \lambda_{2,n,1} + \lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = 0 \\
\text{Node 3: } -\lambda_{3,1,1} - \lambda_{3,2,1} - \cdots - \lambda_{3,n,1} + \lambda_{1,3,1} + \lambda_{2,3,1} + \cdots + \lambda_{n,3,1} = 0 \\
\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots = 0 \\
\text{Node } n\text{: } -\lambda_{n,1,1} - \lambda_{n,2,1} - \lambda_{n,3,1} - \cdots - \lambda_{n,n-1,1} + \lambda_{1,n,1} + \lambda_{2,n,1} + \lambda_{3,n,1} + \cdots + \lambda_{n-1,n,1} = 0 \\
\text{Node 1, outgoing sub-streams: } \lambda_{1,2,1} + \lambda_{1,3,1} + \cdots + \lambda_{1,n,1} = \varphi_{1,2} \\
\text{Node 2, incoming sub-streams: } \lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = \varphi_{1,2}
\end{array}\right.$$

In this system of equations, the values of the sub-streams outgoing from the node are subtracted, and the values of the sub-streams entering the node are added up, the total sum of the sub-streams in the intermediate nodes is zero, the sum of the sub-streams in the source node is equal to the initial data from the array of stream directions and is equal to the sum of the sub-streams in the receiver node.

The creating of this system of incidence equations based on sub-stream matrices consists in the fact that for each node numbered ***i***, sub-streams are subtracted from the row numbered ***i***, and sub-streams are added from the column numbered ***i***. And also for the source node, the outgoing sub-streams are added, and for the receiver node, the incoming sub-streams are added, and this amount is equal to the value of the stream directed from the source node to the receiver node.

For example, consider the formation of an equation for ***node 1***, ***stream*** $\boldsymbol{\varphi_{1,2}}$:

$$\begin{matrix} 0 & \lambda_{1,2,1} & \lambda_{1,3,1} & \cdots & \lambda_{1,n,1} \\ \lambda_{2,1,1} & 0 & \lambda_{2,3,1} & \cdots & \lambda_{2,n,1} \\ \lambda_{3,1,1} & \lambda_{3,2,1} & 0 & \cdots & \lambda_{3,n,1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,1} & \lambda_{n,2,1} & \lambda_{n,3,1} & \cdots & 0 \end{matrix}$$

$$-\lambda_{1,2,1} - \lambda_{1,3,1} - \cdots - \lambda_{1,n,1} + \lambda_{2,1,1} + \lambda_{3,1,1} + \cdots + \lambda_{n,1,1} = 0$$

Outgoing sub-streams are highlighted in turquoise, and incoming ones in purple.
Let's also consider the formation of the equation for ***node 2***:

$$\begin{matrix} 0 & \lambda_{1,2,1} & \lambda_{1,3,1} & \cdots & \lambda_{1,n,1} \\ \lambda_{2,1,1} & 0 & \lambda_{2,3,1} & \cdots & \lambda_{2,n,1} \\ \lambda_{3,1,1} & \lambda_{3,2,1} & 0 & \cdots & \lambda_{3,n,1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,1} & \lambda_{n,2,1} & \lambda_{n,3,1} & \cdots & 0 \end{matrix}$$

$$-\lambda_{2,1,1} - \lambda_{2,3,1} - \cdots - \lambda_{2,n,1} + \lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = 0$$

For the other intermediate nodes, the equations are formed similarly.

Consider the formation of equations for the source node and the receiver node for the ***stream*** $\boldsymbol{\varphi_{1,2}}$:

$$\begin{matrix} 0 & \lambda_{1,2,1} & \lambda_{1,3,1} & \cdots & \lambda_{1,n,1} \\ \lambda_{2,1,1} & 0 & \lambda_{2,3,1} & \cdots & \lambda_{2,n,1} \\ \lambda_{3,1,1} & \lambda_{3,2,1} & 0 & \cdots & \lambda_{3,n,1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n,1,1} & \lambda_{n,2,1} & \lambda_{n,3,1} & \cdots & 0 \end{matrix}$$

Node 1, incoming substreams: $\lambda_{1,2,1} + \lambda_{1,3,1} + \cdots + \lambda_{1,n,1} = \varphi_{1,2}$

Node 2, outgoing substreams: $\lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = \varphi_{1,2}$

For the remaining streams, source nodes and receiver nodes, the equations are formed similarly.

Let's make a general system of incidence equations for streams $\boldsymbol{\varphi_{1,2}}$, $\boldsymbol{\varphi_{1,3}}$, and $\boldsymbol{\varphi_{n,n-1}}$:

$$
\begin{cases}
\begin{cases}
\text{Node 1: } -\lambda_{1,2,1} - \lambda_{1,3,1} - \cdots - \lambda_{1,n,1} + \lambda_{2,1,1} + \lambda_{3,1,1} + \cdots + \lambda_{n,1,1} = 0 \\
\text{Node 2: } -\lambda_{2,1,1} - \lambda_{2,3,1} - \cdots - \lambda_{2,n,1} + \lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = 0 \\
\text{Node 3: } -\lambda_{3,1,1} - \lambda_{3,2,1} - \cdots - \lambda_{3,n,1} + \lambda_{1,3,1} + \lambda_{2,3,1} + \cdots + \lambda_{n,3,1} = 0 \\
\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots = 0 \\
\text{Node } n\text{: } -\lambda_{n,1,1} - \lambda_{n,2,1} - \lambda_{n,3,1} - \cdots - \lambda_{n,n-1,1} + \lambda_{1,n,1} + \lambda_{2,n,1} + \lambda_{3,n,1} + \cdots + \lambda_{n-1,n,1} = 0 \\
\text{Node 1, outgoing sub-streams: } \lambda_{1,2,1} + \lambda_{1,3,1} + \cdots + \lambda_{1,n,1} = \varphi_{1,2} \\
\text{Node 2, incoming sub-streams: } \lambda_{1,2,1} + \lambda_{3,2,1} + \cdots + \lambda_{n,2,1} = \varphi_{1,2}
\end{cases} \\
\begin{cases}
\text{Node 1: } -\lambda_{1,2,2} - \lambda_{1,3,2} - \cdots - \lambda_{1,n,2} + \lambda_{2,1,2} + \lambda_{3,1,2} + \cdots + \lambda_{n,1,2} = 0 \\
\text{Node 2: } -\lambda_{2,1,2} - \lambda_{2,3,2} - \cdots - \lambda_{2,n,2} + \lambda_{1,2,2} + \lambda_{3,2,2} + \cdots + \lambda_{n,2,2} = 0 \\
\text{Node 3: } -\lambda_{3,1,2} - \lambda_{3,2,2} - \cdots - \lambda_{3,n,2} + \lambda_{1,3,2} + \lambda_{2,3,2} + \cdots + \lambda_{n,3,2} = 0 \\
\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots = 0 \\
\text{Node } n\text{: } -\lambda_{n,1,2} - \lambda_{n,2,2} - \lambda_{n,3,2} - \cdots - \lambda_{n,n-1,2} + \lambda_{1,n,2} + \lambda_{2,n,2} + \lambda_{3,n,2} + \cdots + \lambda_{n-1,n,2} = 0 \\
\text{Node 1, outgoing sub-streams: } \lambda_{1,2,2} + \lambda_{1,3,2} + \cdots + \lambda_{1,n,2} = \varphi_{1,3} \\
\text{Node 3, incoming sub-streams: } \lambda_{1,3,2} + \lambda_{2,3,2} + \cdots + \lambda_{n,3,2} = \varphi_{1,3}
\end{cases} \\
\dots\dots\dots \\
\begin{cases}
\text{Node 1: } -\lambda_{1,2,m} - \lambda_{1,3,m} - \cdots - \lambda_{1,n,m} + \lambda_{2,1,m} + \lambda_{3,1,m} + \cdots + \lambda_{n,1,m} = 0 \\
\text{Node 2: } -\lambda_{2,1,m} - \lambda_{2,3,m} - \cdots - \lambda_{2,n,m} + \lambda_{1,2,m} + \lambda_{3,2,m} + \cdots + \lambda_{n,2,m} = 0 \\
\text{Node 3: } -\lambda_{3,1,m} - \lambda_{3,2,m} - \cdots - \lambda_{3,n,m} + \lambda_{1,3,m} + \lambda_{2,3,m} + \cdots + \lambda_{n,3,m} = 0 \\
\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots\dots = 0 \\
\text{Node } n\text{: } -\lambda_{n,1,m} - \lambda_{n,2,m} - \lambda_{n,3,m} - \cdots - \lambda_{n,-1,m} + \lambda_{1,n,m} + \lambda_{2,n,m} + \lambda_{3,n,m} + \cdots + \lambda_{n-1,n,m} = 0 \\
\text{Node } n\text{, outgoing sub-streams: } \lambda_{n,1,m} + \lambda_{n,2,m} + \lambda_{n,3,m} \cdots + \lambda_{n,n-1,m} = \varphi_{n,n-1} \\
\text{Node } n\text{-}1\text{, incoming sub-streams: } \lambda_{1,n-1,m} + \lambda_{2,n-1,m} + \lambda_{3,n-1,m} \dots + \lambda_{n,n-1,m} = \varphi_{n,n-1}
\end{cases}
\end{cases}
$$

where $\boldsymbol{n}$ is the number of nodes in the network, $\boldsymbol{m}$ is the number of streams.

The maximum number of equations in the system is equal to the number of nodes plus 2 multiplied by the number of non-zero streams $\boldsymbol{Q_{eq}=(n+2)\times m}$. The maximum number of variables $\boldsymbol{\lambda}$ equal to the multiplication (of multiplication of the number of nodes by itself minus the number of nodes) by the number of streams $\boldsymbol{Q_{\lambda}=(n^2-n)\times m}$. The maximum number of streams $\boldsymbol{m}$ is also equal to the multiplication of the number of nodes by itself, thus the maximum number of variables $\boldsymbol{Q_{\lambda}=(n^2-n)^2}$.

The equations presented above are constraints for the objective function.

1.4 Objective function

Based on the network structure, a target function is formed. For a convenient representation of the objective function, it is necessary to transform the network structure matrix into a network structure string, the flow matrix into a stream string, and the sub-stream matrices into sub-stream strings, respectively. Each element of the row will match to each element of the matrix, arranged in order line by line. In this way, the indexing of matrix elements will be transformed.

For example, there is a network consisting of 4 nodes, in which 4 streams pass. Then the network matrix will have the following form:

Node numbers	***1***	***2***	***3***	***4***
1	0	$\mu_{1,2}$	$\mu_{1,3}$	$\mu_{1,4}$
2	$\mu_{2,1}$	0	$\mu_{2,3}$	$\mu_{2,4}$
3	$\mu_{3,1}$	$\mu_{3,2}$	0	$\mu_{3,4}$
4	$\mu_{4,1}$	$\mu_{4,2}$	$\mu_{4,3}$	0

The match string will have the following form:

Network structure string	μ_1	μ_2	μ_3	μ_4	μ_5	μ_6	μ_7	μ_8	μ_9	μ_{10}	μ_{11}	μ_{12}
Elements of the network matrix matched to the network structure string	$\mu_{1,2}$	$\mu_{1,3}$	$\mu_{1,4}$	$\mu_{2,1}$	$\mu_{2,3}$	$\mu_{2,4}$	$\mu_{3,1}$	$\mu_{3,2}$	$\mu_{3,4}$	$\mu_{4,1}$	$\mu_{4,2}$	$\mu_{4,3}$

In this string of the network structure, the indexing of elements has been transformed.

Let the streams in this network be distributed as follows: from node 1 to node 4, from node 2 to node 3, from node 3 to node 1, from node 4 to node 2. Then the flow matrix will have the following form:

Node numbers	***1***	***2***	***3***	***4***
1	0	0	0	$\varphi_{1,4}$
2	0	0	$\varphi_{2,3}$	0
3	$\varphi_{3,1}$	0	0	0
4	0	$\varphi_{4,2}$	0	0

The match string will have the following form:

A string of the streams	φ_1	φ_2	φ_3	φ_4
Elements of the flow matrix matched to string of the streams	$\varphi_{1,4}$	$\varphi_{2,3}$	$\varphi_{3,1}$	$\varphi_{4,2}$

Similarly, the indexing of sub-stream matrices for each stream is transformed, while each sub-stream λ in a row will have 2 indexes corresponding to the stream number φ and the element number of the network matrix (bandwidth) μ.

Matrix of sub-streams for the stream φ_1 :

Node numbers	***1***	***2***	***3***	***4***
1	0	$\lambda_{1,2,1}$	$\lambda_{1,3,1}$	$\lambda_{1,4,1}$
2	$\lambda_{2,1,1}$	0	$\lambda_{2,3,1}$	$\lambda_{2,4,1}$
3	$\lambda_{3,1,1}$	$\lambda_{3,2,1}$	0	$\lambda_{3,4,1}$
4	$\lambda_{4,1,1}$	$\lambda_{4,2,1}$	$\lambda_{4,3,1}$	0

The match string of sub-streams for the stream φ_1 :

Network structure string	$\lambda_{1,1}$	$\lambda_{2,1}$	$\lambda_{3,1}$	$\lambda_{4,1}$	$\lambda_{5,1}$	$\lambda_{6,1}$	$\lambda_{7,1}$	$\lambda_{8,1}$	$\lambda_{9,1}$	$\lambda_{10,1}$	$\lambda_{11,1}$	$\lambda_{12,1}$
Elements of the network matrix matched to the network structure string	$\lambda_{1,2,1}$	$\lambda_{1,3,1}$	$\lambda_{1,4,1}$	$\lambda_{2,1,1}$	$\lambda_{2,3,1}$	$\lambda_{2,4,1}$	$\lambda_{3,1,1}$	$\lambda_{3,2,1}$	$\lambda_{3,4,1}$	$\lambda_{4,1,1}$	$\lambda_{4,2,1}$	$\lambda_{4,3,1}$

Sub-stream matrices for streams φ_2, φ_3, φ_4 :

Matrix of sub-streams for streams φ_2 :				
Node numbers	***1***	***2***	***3***	***4***
1	0	$\lambda_{1,2,2}$	$\lambda_{1,3,2}$	$\lambda_{1,4,2}$
2	$\lambda_{2,1,2}$	0	$\lambda_{2,3,2}$	$\lambda_{2,4,2}$
3	$\lambda_{3,1,2}$	$\lambda_{3,2,2}$	0	$\lambda_{3,4,2}$
4	$\lambda_{4,1,2}$	$\lambda_{4,2,2}$	$\lambda_{4,3,2}$	0

Matrix of sub-streams for streams φ_3 :				
Node numbers	***1***	***2***	***3***	***4***
1	0	$\lambda_{1,2,3}$	$\lambda_{1,3,3}$	$\lambda_{1,4,3}$
2	$\lambda_{2,1,3}$	0	$\lambda_{2,3,3}$	$\lambda_{2,4,3}$
3	$\lambda_{3,1,3}$	$\lambda_{3,2,3}$	0	$\lambda_{3,4,3}$
4	$\lambda_{4,1,3}$	$\lambda_{4,2,3}$	$\lambda_{4,3,3}$	0

Matrix of sub-streams for streams φ_4 :				
Node numbers	***1***	***2***	***3***	***4***
1	0	$\lambda_{1,2,4}$	$\lambda_{1,3,4}$	$\lambda_{1,4,4}$
2	$\lambda_{2,1,4}$	0	$\lambda_{2,3,4}$	$\lambda_{2,4,4}$
3	$\lambda_{3,1,4}$	$\lambda_{3,2,4}$	0	$\lambda_{3,4,4}$
4	$\lambda_{4,1,4}$	$\lambda_{4,2,4}$	$\lambda_{4,3,4}$	0

Substream strings for streams matched to sub-streams matrices φ_2, φ_3, φ_4 :

The string of the network structure for the stream φ_2	$\lambda_{1,2}$	$\lambda_{2,2}$	$\lambda_{3,2}$	$\lambda_{4,2}$	$\lambda_{5,2}$	$\lambda_{6,2}$	$\lambda_{7,2}$	$\lambda_{8,2}$	$\lambda_{9,2}$	$\lambda_{10,2}$	$\lambda_{11,2}$	$\lambda_{12,2}$
The string of the network structure for the stream φ_3	$\lambda_{1,3}$	$\lambda_{2,3}$	$\lambda_{3,3}$	$\lambda_{4,3}$	$\lambda_{5,3}$	$\lambda_{6,3}$	$\lambda_{7,3}$	$\lambda_{8,3}$	$\lambda_{9,3}$	$\lambda_{10,3}$	$\lambda_{11,3}$	$\lambda_{12,3}$
The string of the network structure for the stream φ_4	$\lambda_{1,4}$	$\lambda_{2,4}$	$\lambda_{3,4}$	$\lambda_{4,4}$	$\lambda_{5,4}$	$\lambda_{6,4}$	$\lambda_{7,4}$	$\lambda_{8,4}$	$\lambda_{9,4}$	$\lambda_{10,4}$	$\lambda_{11,4}$	$\lambda_{12,4}$

Having a convenient indexing of elements, an objective function for the information network is formed from the above example:

$$N = \frac{\frac{\lambda_{1,1} + \lambda_{1,2} + \lambda_{1,3} + \lambda_{1,4}}{\mu_1}}{1 - \frac{\lambda_{1,1} + \lambda_{1,2} + \lambda_{1,3} + \lambda_{1,4}}{\mu_1}} + \frac{\frac{\lambda_{2,1} + \lambda_{2,2} + \lambda_{2,3} + \lambda_{2,4}}{\mu_2}}{1 - \frac{\lambda_{2,1} + \lambda_{2,2} + \lambda_{2,3} + \lambda_{2,4}}{\mu_2}} + \frac{\frac{\lambda_{3,1} + \lambda_{3,2} + \lambda_{3,3} + \lambda_{3,4}}{\mu_3}}{1 - \frac{\lambda_{3,1} + \lambda_{3,2} + \lambda_{3,3} + \lambda_{3,4}}{\mu_3}} +$$

$$+ \frac{\frac{\lambda_{4,1} + \lambda_{4,2} + \lambda_{4,3} + \lambda_{4,4}}{\mu_4}}{1 - \frac{\lambda_{3,1} + \lambda_{3,2} + \lambda_{3,3} + \lambda_{3,4}}{\mu_4}} + \frac{\frac{\lambda_{5,1} + \lambda_{5,2} + \lambda_{5,3} + \lambda_{5,4}}{\mu_5}}{1 - \frac{\lambda_{3,1} + \lambda_{3,2} + \lambda_{3,3} + \lambda_{3,4}}{\mu_5}} + \frac{\frac{\lambda_{6,1} + \lambda_{6,2} + \lambda_{6,3} + \lambda_{6,4}}{\mu_6}}{1 - \frac{\lambda_{6,1} + \lambda_{6,2} + \lambda_{6,3} + \lambda_{6,4}}{\mu_6}} +$$

$$+ \frac{\frac{\lambda_{7,1} + \lambda_{7,2} + \lambda_{7,3} + \lambda_{7,4}}{\mu_7}}{1 - \frac{\lambda_{7,1} + \lambda_{7,2} + \lambda_{7,3} + \lambda_{7,4}}{\mu_7}} + \frac{\frac{\lambda_{8,1} + \lambda_{8,2} + \lambda_{8,3} + \lambda_{8,4}}{\mu_8}}{1 - \frac{\lambda_{8,1} + \lambda_{8,2} + \lambda_{8,3} + \lambda_{8,4}}{\mu_8}} + \frac{\frac{\lambda_{9,1} + \lambda_{9,2} + \lambda_{9,3} + \lambda_{9,4}}{\mu_9}}{1 - \frac{\lambda_{3,1} + \lambda_{3,2} + \lambda_{3,3} + \lambda_{3,4}}{\mu_9}} +$$

$$+ \frac{\frac{\lambda_{10,1} + \lambda_{10,2} + \lambda_{10,3} + \lambda_{10,4}}{\mu_{10}}}{1 - \frac{\lambda_{10,1} + \lambda_{10,2} + \lambda_{10,3} + \lambda_{10,4}}{\mu_{10}}} + \frac{\frac{\lambda_{11,1} + \lambda_{11,2} + \lambda_{11,3} + \lambda_{11,4}}{\mu_{11}}}{1 - \frac{\lambda_{11,1} + \lambda_{11,2} + \lambda_{11,3} + \lambda_{11,4}}{\mu_{11}}} + \frac{\frac{\lambda_{12,1} + \lambda_{12,2} + \lambda_{12,3} + \lambda_{12,4}}{\mu_{12}}}{1 - \frac{\lambda_{12,1} + \lambda_{12,2} + \lambda_{12,3} + \lambda_{12,4}}{\mu_{12}}}.$$

Thus, the objective function for an information network consisting of ***n*** communication channels and ***m*** streams has the following form:

$$N(\lambda) = \sum_{i=1}^{n} \frac{\frac{\sum_{j=1}^{m} \lambda_{i,j}}{\mu_i}}{1 - \frac{\sum_{j=1}^{m} \lambda_{i,j}}{\mu_i}},$$

where ***N*** is the average queue of threads, ***i*** is the communication channel number, ***n*** is the number of communication channels, j is the stream number, m is the number of threads, μ_i is the amount of bandwidth of the ***i***-th communication channel, $\lambda_{i,j}$ is the searchable value of the sub-stream of the ***j***-th stream in the ***i***-th communication channel.

The problem of optimizing information streams in data transmission networks is reduced to finding such values of $\lambda_{i,j}$, so that the value of the average queue of streams ***N*** is the minimum possible, taking into account the limitations of the incidence equations. This problem is solved using one of the optimization algorithms, for example, the algorithm of sequential quadratic programming. Software algorithms for optimizing the objective function with constraints have been around for a long time, however, a feature of the presented algorithm is the automatic creating of the objective function and constraints to it based on the adjacency matrix of the network and the stream matrix, as well as the automatic formation of matrices of optimal stream distributions. In the presented software algorithm, optimization will be performed using the ***fmincon*** function from the Matlab software package library.

2. Optimization calculation example

2.1 Initial data

Let the information network have the following structure:

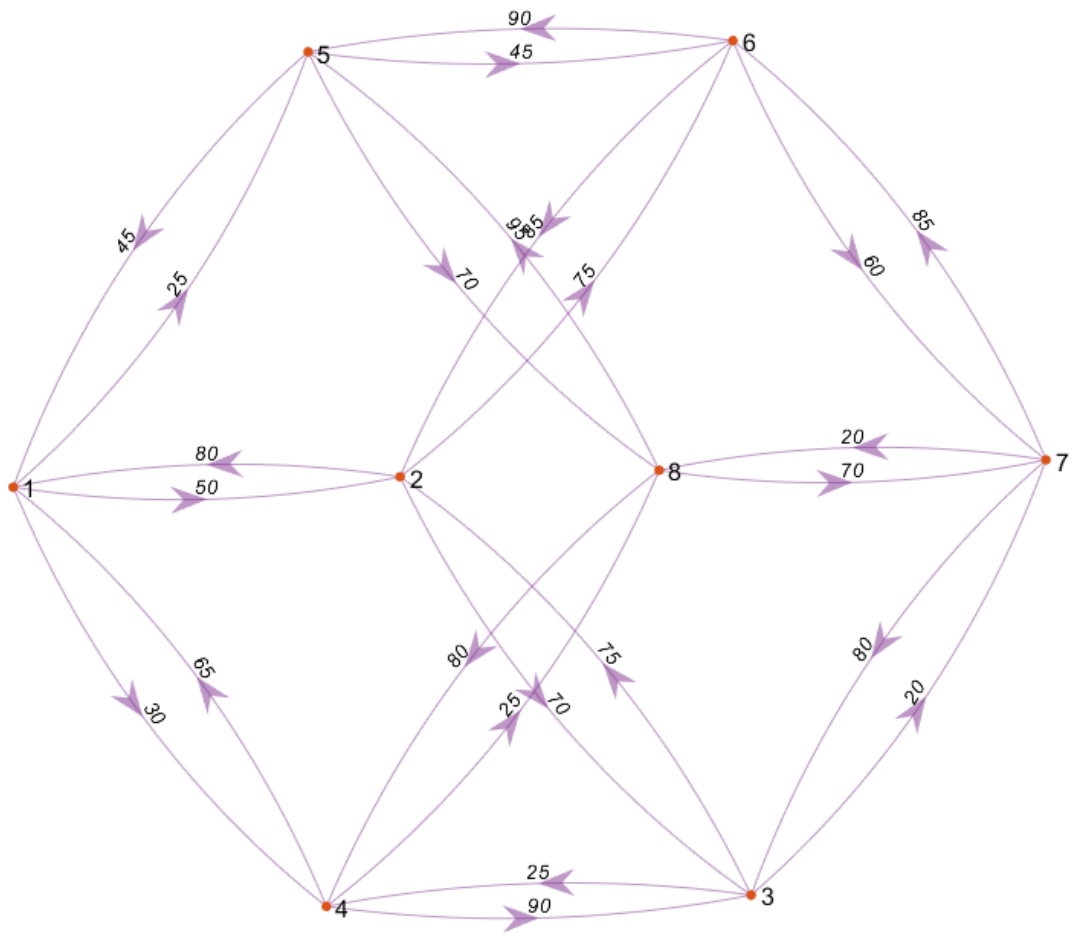

Figure 2.1.1 – graph of the information network with indication of node numbers, node adjacency, directions and bandwidth of communication channels.

Then the adjacency matrix of this network has the following form:

		Node numbers							
		1	2	3	4	5	6	7	8
Node numbers	1	0	50	0	30	25	0	0	0
	2	80	0	70	0	0	75	0	0
	3	0	75	0	25	0	0	20	0
	4	65	0	90	0	0	0	0	25
	5	45	0	0	0	0	45	0	70
	6	0	85	0	0	90	0	60	0
	7	0	0	80	0	0	85	0	20
	8	0	0	0	80	95	0	70	0

This network consists of eight nodes and twenty-four communication channels.

Let 3 streams with the following directions and values pass through this network at the current time: from ***node 1*** to ***node 7*** of value ***78***, from ***node 3*** to ***node 5*** of value ***48***, from ***node 6*** to ***node 4*** of value ***61***.

Then the matrix of streams of this network has the following form:

		Node numbers							
		1	2	3	4	5	6	7	8
Node numbers	1	0	0	0	0	0	0	**78**	0
	2	0	0	0	0	0	0	0	0
	3	0	0	0	0	**48**	0	0	0
	4	0	0	0	0	0	0	0	0
	5	0	0	0	0	0	0	0	0
	6	0	0	0	**61**	0	0	0	0
	7	0	0	0	0	0	0	0	0
	8	0	0	0	0	0	0	0	0

2.2 Objective function limitations

Based on the adjacency matrix , an incidence matrix is formed for each communication channel:

		Indexes of communication channels																								
		$\lambda_{1,2}$	$\lambda_{1,4}$	$\lambda_{1,5}$	$\lambda_{2,1}$	$\lambda_{2,3}$	$\lambda_{2,6}$	$\lambda_{3,2}$	$\lambda_{3,4}$	$\lambda_{3,7}$	$\lambda_{4,1}$	$\lambda_{4,3}$	$\lambda_{4,8}$	$\lambda_{5,1}$	$\lambda_{5,6}$	$\lambda_{5,8}$	$\lambda_{6,2}$	$\lambda_{6,5}$	$\lambda_{6,7}$	$\lambda_{7,3}$	$\lambda_{7,6}$	$\lambda_{7,8}$	$\lambda_{8,4}$	$\lambda_{8,5}$	$\lambda_{8,7}$
Node numbers	***1***	-1	-1	-1	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0
	2	1	0	0	-1	-1	-1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
	3	0	0	0	0	1	0	-1	-1	-1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0
	4	0	1	0	0	0	0	0	1	0	-1	-1	-1	0	0	0	0	0	0	0	0	0	1	0	0
	5	0	0	1	0	0	0	0	0	0	0	0	0	-1	-1	-1	0	1	0	0	0	0	0	1	0
	6	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	-1	-1	-1	0	1	0	0	0	0
	7	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	-1	-1	-1	0	0	1
	8	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	1	-1	-1	-1

Multiplying each value of λ by the value of the incidence matrix, a system of flow equations for each node is compiled in general form:

$$\begin{cases} 1: -\lambda_{1,2} - \lambda_{1,4} - \lambda_{1,5} + \lambda_{2,1} + \lambda_{4,1} + \lambda_{5,1} = 0; \\ 2: \lambda_{1,2} - \lambda_{2,1} - \lambda_{2,3} - \lambda_{2,6} + \lambda_{3,2} + \lambda_{6,2} = 0; \\ 3: \lambda_{2,3} - \lambda_{3,2} - \lambda_{3,4} - \lambda_{3,7} + \lambda_{4,3} + \lambda_{7,3} = 0; \\ 4: \lambda_{1,4} + \lambda_{3,4} - \lambda_{4,1} - \lambda_{4,3} - \lambda_{4,8} + \lambda_{8,4} = 0; \\ 5: \lambda_{1,5} - \lambda_{5,1} - \lambda_{5,6} - \lambda_{5,8} + \lambda_{6,5} + \lambda_{8,5} = 0; \\ 6: \lambda_{2,6} + \lambda_{5,6} - \lambda_{6,2} - \lambda_{6,5} - \lambda_{6,7} + \lambda_{7,6} = 0; \\ 7: \lambda_{3,7} + \lambda_{6,7} - \lambda_{7,3} - \lambda_{7,6} - \lambda_{7,8} + \lambda_{8,7} = 0; \\ 8: \lambda_{4,8} + \lambda_{5,8} + \lambda_{7,8} - \lambda_{8,4} - \lambda_{8,5} - \lambda_{8,7} = 0. \end{cases}$$

For convenience of display, the names of the streams outgoing from the node are highlighted in turquoise, and the names of the streams incoming to the node are highlighted in purple. In the above system of equations, each value of λ_{ij} is the sum of the sub-streams passing through the communication channel directed from node ***i*** to node ***j***.

Equating the sum of the outgoing sub-streams of the source node and the sum of the incoming sub-streams of the receiver node to the value of the specified flows, and the sum of the incoming sub-streams of the source node and the sum of the outgoing sub-streams of the receiver node to zero, we will compile

systems of sub-stream equations for each stream. In systems of sub-stream equations, a third index corresponding to the stream number is added to the names λ.

The system of sub-streams equations for the stream from node 1 to node 7:

$$\begin{cases} 1: \lambda_{2,1,1} + \lambda_{4,1,1} + \lambda_{5,1,1} = 0; \\ 2: \lambda_{1,2,1} - \lambda_{2,1,1} - \lambda_{2,3,1} - \lambda_{2,6,1} + \lambda_{3,2,1} + \lambda_{6,2,1} = 0; \\ 3: \lambda_{2,3,1} - \lambda_{3,2,1} - \lambda_{3,4,1} - \lambda_{3,7,1} + \lambda_{4,3,1} + \lambda_{7,3,1} = 0; \\ 4: \lambda_{1,4,1} + \lambda_{3,4,1} - \lambda_{4,1,1} - \lambda_{4,3,1} - \lambda_{4,8,1} + \lambda_{8,4,1} = 0; \\ 5: \lambda_{1,5,1} - \lambda_{5,1,1} - \lambda_{5,6,1} - \lambda_{5,8,1} + \lambda_{6,5,1} + \lambda_{8,5,1} = 0; \\ 6: \lambda_{2,6,1} + \lambda_{5,6,1} - \lambda_{6,2,1} - \lambda_{6,5,1} - \lambda_{6,7,1} + \lambda_{7,6,1} = 0; \\ 7: -\lambda_{7,3,1} - \lambda_{7,6,1} - \lambda_{7,8,1} = 0; \\ 8: \lambda_{4,8,1} + \lambda_{5,8,1} + \lambda_{7,8,1} - \lambda_{8,4,1} - \lambda_{8,5,1} - \lambda_{8,7,1} = 0; \\ 1: -\lambda_{1,2,1} - \lambda_{1,4,1} - \lambda_{1,5,1} = -78; \\ 7: \lambda_{3,7,1} + \lambda_{6,7,1} + \lambda_{8,7,1} = 78. \end{cases}$$

The system of sub-streams equations for the stream from node 3 to node 5:

$$\begin{cases} 1: -\lambda_{1,2,2} - \lambda_{1,4,2} - \lambda_{1,5,2} + \lambda_{2,1,2} + \lambda_{4,1,2} + \lambda_{5,1,2} = 0; \\ 2: \lambda_{1,2,2} - \lambda_{2,1,2} - \lambda_{2,3,2} - \lambda_{2,6,2} + \lambda_{3,2,2} + \lambda_{6,2,2} = 0; \\ 3: \lambda_{2,3,2} + \lambda_{4,3,2} + \lambda_{7,3,2} = 0; \\ 4: \lambda_{1,4,2} + \lambda_{3,4,2} - \lambda_{4,1,2} - \lambda_{4,3,2} - \lambda_{4,8,2} + \lambda_{8,4,2} = 0; \\ 5: \lambda_{5,1,2} - \lambda_{5,6,2} - \lambda_{5,8,2} = 0; \\ 6: \lambda_{2,6,2} + \lambda_{5,6,2} - \lambda_{6,2,2} - \lambda_{6,5,2} - \lambda_{6,7,2} + \lambda_{7,6,2} = 0; \\ 7: \lambda_{3,7,2} + \lambda_{6,7,2} - \lambda_{7,3,2} - \lambda_{7,6,2} - \lambda_{7,8,2} + \lambda_{8,7,2} = 0; \\ 8: \lambda_{4,8,2} + \lambda_{5,8,2} + \lambda_{7,8,2} - \lambda_{8,4,2} - \lambda_{8,5,2} - \lambda_{8,7,2} = 0; \\ 3: -\lambda_{3,2,2} - \lambda_{3,4,2} - \lambda_{3,7,2} = -48; \\ 5: \lambda_{1,5,2} + \lambda_{6,5,2} + \lambda_{8,5,2} = 48. \end{cases}$$

The system of sub-streams equations for the stream from node 6 to node 4:

$$\begin{cases} 1: -\lambda_{1,2,3} - \lambda_{1,4,3} - \lambda_{1,5,3} + \lambda_{2,1,3} + \lambda_{4,1,3} + \lambda_{5,1,3} = 0; \\ 2: \lambda_{1,2,3} - \lambda_{2,1,3} - \lambda_{2,3,3} - \lambda_{2,6,3} + \lambda_{3,2,3} + \lambda_{6,2,3} = 0; \\ 3: \lambda_{2,3,3} - \lambda_{3,2,3} - \lambda_{3,4,3} - \lambda_{3,7,3} + \lambda_{4,3,3} + \lambda_{7,3,3} = 0; \\ 4: -\lambda_{4,1,3} - \lambda_{4,3,3} - \lambda_{4,8,3} = 0; \\ 5: \lambda_{1,5,3} - \lambda_{5,1,3} - \lambda_{5,6,3} - \lambda_{5,8,3} + \lambda_{6,5,3} + \lambda_{8,5,3} = 0; \\ 6: \lambda_{2,6,3} + \lambda_{5,6,3} + \lambda_{7,6,3} = 0; \\ 7: \lambda_{3,7,3} + \lambda_{6,7,3} - \lambda_{7,3,3} - \lambda_{7,6,3} - \lambda_{7,8,3} + \lambda_{8,7,3} = 0; \\ 8: \lambda_{4,8,3} + \lambda_{5,8,3} + \lambda_{7,8,3} - \lambda_{8,4,3} - \lambda_{8,5,3} - \lambda_{8,7,3} = 0; \\ 6: -\lambda_{6,2,3} - \lambda_{6,5,3} - \lambda_{6,7,3} = -61; \\ 4: \lambda_{1,4,3} + \lambda_{3,4,3} + \lambda_{8,4,3} = 61. \end{cases}$$

Based on the adjacency matrix, a table has been compiled with the indices of communication channels and their corresponding values of bandwidth $\boldsymbol{\mu_{ij}}$:

$\mu_{1,2}$	$\mu_{1,4}$	$\mu_{1,5}$	$\mu_{2,1}$	$\mu_{2,3}$	$\mu_{2,6}$	$\mu_{3,2}$	$\mu_{3,4}$	$\mu_{3,7}$	$\mu_{4,1}$	$\mu_{4,3}$	$\mu_{4,8}$	$\mu_{5,1}$	$\mu_{5,6}$	$\mu_{5,8}$	$\mu_{6,2}$	$\mu_{6,5}$	$\mu_{6,7}$	$\mu_{7,3}$	$\mu_{7,6}$	$\mu_{7,8}$	$\mu_{8,4}$	$\mu_{8,5}$	$\mu_{8,7}$
50	30	25	80	70	75	75	25	20	65	90	25	45	45	70	85	90	60	80	85	20	80	95	70

A system of inequalities has been compiled that limits the sum of the intensities of sub-streams in communication channels to the value of bandwidth of these communication channels:

$$\begin{cases}
\lambda_{1,2,1} + \lambda_{1,2,2} + \lambda_{1,2,3} \leq \mu_{1,2}; \\
\lambda_{1,4,1} + \lambda_{1,4,2} + \lambda_{1,4,3} \leq \mu_{1,4}; \\
\lambda_{1,5,1} + \lambda_{1,5,2} + \lambda_{1,5,3} \leq \mu_{1,5}; \\
\lambda_{2,1,1} + \lambda_{2,1,2} + \lambda_{2,1,3} \leq \mu_{2,1}; \\
\lambda_{2,3,1} + \lambda_{2,3,2} + \lambda_{2,3,3} \leq \mu_{2,3}; \\
\lambda_{2,6,1} + \lambda_{2,6,2} + \lambda_{2,6,3} \leq \mu_{2,6}; \\
\lambda_{3,2,1} + \lambda_{3,2,2} + \lambda_{3,2,3} \leq \mu_{3,2}; \\
\lambda_{3,4,1} + \lambda_{3,4,2} + \lambda_{3,4,3} \leq \mu_{3,4}; \\
\lambda_{3,7,1} + \lambda_{3,7,2} + \lambda_{3,7,3} \leq \mu_{3,7}; \\
\lambda_{4,1,1} + \lambda_{4,1,2} + \lambda_{4,1,3} \leq \mu_{4,1}; \\
\lambda_{4,3,1} + \lambda_{4,3,2} + \lambda_{4,3,3} \leq \mu_{4,3}; \\
\lambda_{4,8,1} + \lambda_{4,8,2} + \lambda_{4,8,3} \leq \mu_{4,8}; \\
\lambda_{5,1,1} + \lambda_{5,1,2} + \lambda_{5,1,3} \leq \mu_{5,1}; \\
\lambda_{5,6,1} + \lambda_{5,6,2} + \lambda_{5,6,3} \leq \mu_{5,6}; \\
\lambda_{5,8,1} + \lambda_{5,8,2} + \lambda_{5,8,3} \leq \mu_{5,8}; \\
\lambda_{6,2,1} + \lambda_{6,2,2} + \lambda_{6,2,3} \leq \mu_{6,2}; \\
\lambda_{6,5,1} + \lambda_{6,5,2} + \lambda_{6,5,3} \leq \mu_{6,5}; \\
\lambda_{6,7,1} + \lambda_{6,7,2} + \lambda_{6,7,3} \leq \mu_{6,7}; \\
\lambda_{7,3,1} + \lambda_{7,3,2} + \lambda_{7,3,3} \leq \mu_{7,3}; \\
\lambda_{7,6,1} + \lambda_{7,6,2} + \lambda_{7,6,3} \leq \mu_{7,6}; \\
\lambda_{7,8,1} + \lambda_{7,8,2} + \lambda_{7,8,3} \leq \mu_{7,8}; \\
\lambda_{8,4,1} + \lambda_{8,4,2} + \lambda_{8,4,3} \leq \mu_{8,4}; \\
\lambda_{8,5,1} + \lambda_{8,5,2} + \lambda_{8,5,3} \leq \mu_{8,5}; \\
\lambda_{8,7,1} + \lambda_{8,7,2} + \lambda_{8,7,3} \leq \mu_{8,7}.
\end{cases}$$

Numerical values of bandwinth are substituted into a system of inequalities:

$$\begin{cases}
\lambda_{1,2,1} + \lambda_{1,2,2} + \lambda_{1,2,3} \leq 50; \\
\lambda_{1,4,1} + \lambda_{1,4,2} + \lambda_{1,4,3} \leq 30; \\
\lambda_{1,5,1} + \lambda_{1,5,2} + \lambda_{1,5,3} \leq 25; \\
\lambda_{2,1,1} + \lambda_{2,1,2} + \lambda_{2,1,3} \leq 80; \\
\lambda_{2,3,1} + \lambda_{2,3,2} + \lambda_{2,3,3} \leq 70; \\
\lambda_{2,6,1} + \lambda_{2,6,2} + \lambda_{2,6,3} \leq 75; \\
\lambda_{3,2,1} + \lambda_{3,2,2} + \lambda_{3,2,3} \leq 75; \\
\lambda_{3,4,1} + \lambda_{3,4,2} + \lambda_{3,4,3} \leq 25; \\
\lambda_{3,7,1} + \lambda_{3,7,2} + \lambda_{3,7,3} \leq 20; \\
\lambda_{4,1,1} + \lambda_{4,1,2} + \lambda_{4,1,3} \leq 65; \\
\lambda_{4,3,1} + \lambda_{4,3,2} + \lambda_{4,3,3} \leq 90; \\
\lambda_{4,8,1} + \lambda_{4,8,2} + \lambda_{4,8,3} \leq 25; \\
\lambda_{5,1,1} + \lambda_{5,1,2} + \lambda_{5,1,3} \leq 45; \\
\lambda_{5,6,1} + \lambda_{5,6,2} + \lambda_{5,6,3} \leq 45; \\
\lambda_{5,8,1} + \lambda_{5,8,2} + \lambda_{5,8,3} \leq 70; \\
\lambda_{6,2,1} + \lambda_{6,2,2} + \lambda_{6,2,3} \leq 85; \\
\lambda_{6,5,1} + \lambda_{6,5,2} + \lambda_{6,5,3} \leq 90; \\
\lambda_{6,7,1} + \lambda_{6,7,2} + \lambda_{6,7,3} \leq 60; \\
\lambda_{7,3,1} + \lambda_{7,3,2} + \lambda_{7,3,3} \leq 80; \\
\lambda_{7,6,1} + \lambda_{7,6,2} + \lambda_{7,6,3} \leq 85; \\
\lambda_{7,8,1} + \lambda_{7,8,2} + \lambda_{7,8,3} \leq 20; \\
\lambda_{8,4,1} + \lambda_{8,4,2} + \lambda_{8,4,3} \leq 80; \\
\lambda_{8,5,1} + \lambda_{8,5,2} + \lambda_{8,5,3} \leq 95; \\
\lambda_{8,7,1} + \lambda_{8,7,2} + \lambda_{8,7,3} \leq 70.
\end{cases}$$

2.3 Objective function

Based on the formula of the average queue, the target function for this information system is compiled:

$$
\begin{aligned}
N(\lambda_{i,j,k}) = {} & \frac{\frac{\lambda_{1,2,1}+\lambda_{1,2,2}+\lambda_{1,2,3}}{\mu_{1,2}}}{1-\frac{\lambda_{1,2,1}+\lambda_{1,2,2}+\lambda_{1,2,3}}{\mu_{1,2}}} + \frac{\frac{\lambda_{1,4,1}+\lambda_{1,4,2}+\lambda_{1,4,3}}{\mu_{1,4}}}{1-\frac{\lambda_{1,4,1}+\lambda_{1,4,2}+\lambda_{1,4,3}}{\mu_{1,4}}} + \frac{\frac{\lambda_{1,5,1}+\lambda_{1,5,2}+\lambda_{1,5,3}}{\mu_{1,5}}}{1-\frac{\lambda_{1,5,1}+\lambda_{1,5,2}+\lambda_{1,5,3}}{\mu_{1,5}}} \\
& + \frac{\frac{\lambda_{2,1,1}+\lambda_{2,1,2}+\lambda_{2,1,3}}{\mu_{2,1}}}{1-\frac{\lambda_{2,1,1}+\lambda_{2,1,2}+\lambda_{2,1,3}}{\mu_{2,1}}} + \frac{\frac{\lambda_{2,3,1}+\lambda_{2,3,2}+\lambda_{2,3,3}}{\mu_{2,3}}}{1-\frac{\lambda_{2,3,1}+\lambda_{2,3,2}+\lambda_{2,3,3}}{\mu_{2,3}}} + \frac{\frac{\lambda_{2,6,1}+\lambda_{2,6,2}+\lambda_{2,6,3}}{\mu_{2,6}}}{1-\frac{\lambda_{2,6,1}+\lambda_{2,6,2}+\lambda_{2,6,3}}{\mu_{2,6}}} \\
& + \frac{\frac{\lambda_{3,2,1}+\lambda_{3,2,2}+\lambda_{3,2,3}}{\mu_{3,2}}}{1-\frac{\lambda_{3,2,1}+\lambda_{3,2,2}+\lambda_{3,2,3}}{\mu_{3,2}}} + \frac{\frac{\lambda_{3,4,1}+\lambda_{3,4,2}+\lambda_{3,4,3}}{\mu_{3,4}}}{1-\frac{\lambda_{3,4,1}+\lambda_{3,4,2}+\lambda_{3,4,3}}{\mu_{3,4}}} + \frac{\frac{\lambda_{3,7,1}+\lambda_{3,7,2}+\lambda_{3,7,3}}{\mu_{3,7}}}{1-\frac{\lambda_{3,7,1}+\lambda_{3,7,2}+\lambda_{3,7,3}}{\mu_{3,7}}} \\
& + \frac{\frac{\lambda_{4,1,1}+\lambda_{4,1,2}+\lambda_{4,1,3}}{\mu_{4,1}}}{1-\frac{\lambda_{4,1,1}+\lambda_{4,1,2}+\lambda_{4,1,3}}{\mu_{4,1}}} + \frac{\frac{\lambda_{4,3,1}+\lambda_{4,3,2}+\lambda_{4,3,3}}{\mu_{4,3}}}{1-\frac{\lambda_{4,3,1}+\lambda_{4,3,2}+\lambda_{4,3,3}}{\mu_{4,3}}} + \frac{\frac{\lambda_{4,8,1}+\lambda_{4,8,2}+\lambda_{4,8,3}}{\mu_{4,8}}}{1-\frac{\lambda_{4,8,1}+\lambda_{4,8,2}+\lambda_{4,8,3}}{\mu_{4,8}}} \\
& + \frac{\frac{\lambda_{5,1,1}+\lambda_{5,1,2}+\lambda_{5,1,3}}{\mu_{5,1}}}{1-\frac{\lambda_{5,1,1}+\lambda_{5,1,2}+\lambda_{5,1,3}}{\mu_{5,1}}} + \frac{\frac{\lambda_{5,6,1}+\lambda_{5,6,2}+\lambda_{5,6,3}}{\mu_{5,6}}}{1-\frac{\lambda_{5,6,1}+\lambda_{5,6,2}+\lambda_{5,6,3}}{\mu_{5,6}}} + \frac{\frac{\lambda_{5,8,1}+\lambda_{5,8,2}+\lambda_{5,8,3}}{\mu_{5,8}}}{1-\frac{\lambda_{5,8,1}+\lambda_{5,8,2}+\lambda_{5,8,3}}{\mu_{5,8}}} \\
& + \frac{\frac{\lambda_{6,2,1}+\lambda_{6,2,2}+\lambda_{6,2,3}}{\mu_{6,2}}}{1-\frac{\lambda_{6,2,1}+\lambda_{6,2,2}+\lambda_{6,2,3}}{\mu_{6,2}}} + \frac{\frac{\lambda_{6,5,1}+\lambda_{6,5,2}+\lambda_{6,5,3}}{\mu_{6,5}}}{1-\frac{\lambda_{6,5,1}+\lambda_{6,5,2}+\lambda_{6,5,3}}{\mu_{6,5}}} + \frac{\frac{\lambda_{6,7,1}+\lambda_{6,7,2}+\lambda_{6,7,3}}{\mu_{6,7}}}{1-\frac{\lambda_{6,7,1}+\lambda_{6,7,2}+\lambda_{6,7,3}}{\mu_{6,7}}} \\
& + \frac{\frac{\lambda_{7,3,1}+\lambda_{7,3,2}+\lambda_{7,3,3}}{\mu_{7,3}}}{1-\frac{\lambda_{7,3,1}+\lambda_{7,3,2}+\lambda_{7,3,3}}{\mu_{7,3}}} + \frac{\frac{\lambda_{7,6,1}+\lambda_{7,6,2}+\lambda_{7,6,3}}{\mu_{7,6}}}{1-\frac{\lambda_{7,6,1}+\lambda_{7,6,2}+\lambda_{7,6,3}}{\mu_{7,6}}} + \frac{\frac{\lambda_{7,8,1}+\lambda_{7,8,2}+\lambda_{7,8,3}}{\mu_{7,8}}}{1-\frac{\lambda_{7,8,1}+\lambda_{7,8,2}+\lambda_{7,8,3}}{\mu_{7,8}}} \\
& + \frac{\frac{\lambda_{8,4,1}+\lambda_{8,4,2}+\lambda_{8,4,3}}{\mu_{8,4}}}{1-\frac{\lambda_{8,4,1}+\lambda_{8,4,2}+\lambda_{8,4,3}}{\mu_{8,4}}} + \frac{\frac{\lambda_{8,5,1}+\lambda_{8,5,2}+\lambda_{8,5,3}}{\mu_{8,5}}}{1-\frac{\lambda_{8,5,1}+\lambda_{8,5,2}+\lambda_{8,5,3}}{\mu_{8,5}}} + \frac{\frac{\lambda_{8,7,1}+\lambda_{8,7,2}+\lambda_{8,7,3}}{\mu_{8,7}}}{1-\frac{\lambda_{8,7,1}+\lambda_{8,7,2}+\lambda_{8,7,3}}{\mu_{8,7}}},
\end{aligned}
$$

where $\boldsymbol{\lambda_{i,j,k}}$ is the searchable optimal intensity of the sub-stream directed from ***node i*** to ***node j***, which is part of the ***stream k***.

Numerical values of bandwidth are substituted into the objective function:

$$N(\lambda_{i,j,k}) = \frac{\frac{\lambda_{1,2,1}+\lambda_{1,2,2}+\lambda_{1,2,3}}{50}}{1-\frac{\lambda_{1,2,1}+\lambda_{1,2,2}+\lambda_{1,2,3}}{50}} + \frac{\frac{\lambda_{1,4,1}+\lambda_{1,4,2}+\lambda_{1,4,3}}{30}}{1-\frac{\lambda_{1,4,1}+\lambda_{1,4,2}+\lambda_{1,4,3}}{30}} + \frac{\frac{\lambda_{1,5,1}+\lambda_{1,5,2}+\lambda_{1,5,3}}{25}}{1-\frac{\lambda_{1,5,1}+\lambda_{1,5,2}+\lambda_{1,5,3}}{25}}$$
$$+\frac{\frac{\lambda_{2,1,1}+\lambda_{2,1,2}+\lambda_{2,1,3}}{80}}{1-\frac{\lambda_{2,1,1}+\lambda_{2,1,2}+\lambda_{2,1,3}}{80}} + \frac{\frac{\lambda_{2,3,1}+\lambda_{2,3,2}+\lambda_{2,3,3}}{70}}{1-\frac{\lambda_{2,3,1}+\lambda_{2,3,2}+\lambda_{2,3,3}}{70}} + \frac{\frac{\lambda_{2,6,1}+\lambda_{2,6,2}+\lambda_{2,6,3}}{75}}{1-\frac{\lambda_{2,6,1}+\lambda_{2,6,2}+\lambda_{2,6,3}}{75}}$$
$$+\frac{\frac{\lambda_{3,2,1}+\lambda_{3,2,2}+\lambda_{3,2,3}}{75}}{1-\frac{\lambda_{3,2,1}+\lambda_{3,2,2}+\lambda_{3,2,3}}{75}} + \frac{\frac{\lambda_{3,4,1}+\lambda_{3,4,2}+\lambda_{3,4,3}}{25}}{1-\frac{\lambda_{3,4,1}+\lambda_{3,4,2}+\lambda_{3,4,3}}{25}} + \frac{\frac{\lambda_{3,7,1}+\lambda_{3,7,2}+\lambda_{3,7,3}}{20}}{1-\frac{\lambda_{3,7,1}+\lambda_{3,7,2}+\lambda_{3,7,3}}{20}}$$
$$+\frac{\frac{\lambda_{4,1,1}+\lambda_{4,1,2}+\lambda_{4,1,3}}{65}}{1-\frac{\lambda_{4,1,1}+\lambda_{4,1,2}+\lambda_{4,1,3}}{65}} + \frac{\frac{\lambda_{4,3,1}+\lambda_{4,3,2}+\lambda_{4,3,3}}{90}}{1-\frac{\lambda_{4,3,1}+\lambda_{4,3,2}+\lambda_{4,3,3}}{90}} + \frac{\frac{\lambda_{4,8,1}+\lambda_{4,8,2}+\lambda_{4,8,3}}{25}}{1-\frac{\lambda_{4,8,1}+\lambda_{4,8,2}+\lambda_{4,8,3}}{25}}$$
$$+\frac{\frac{\lambda_{5,1,1}+\lambda_{5,1,2}+\lambda_{5,1,3}}{45}}{1-\frac{\lambda_{5,1,1}+\lambda_{5,1,2}+\lambda_{5,1,3}}{45}} + \frac{\frac{\lambda_{5,6,1}+\lambda_{5,6,2}+\lambda_{5,6,3}}{45}}{1-\frac{\lambda_{5,6,1}+\lambda_{5,6,2}+\lambda_{5,6,3}}{45}} + \frac{\frac{\lambda_{5,8,1}+\lambda_{5,8,2}+\lambda_{5,8,3}}{70}}{1-\frac{\lambda_{5,8,1}+\lambda_{5,8,2}+\lambda_{5,8,3}}{70}}$$
$$+\frac{\frac{\lambda_{6,2,1}+\lambda_{6,2,2}+\lambda_{6,2,3}}{85}}{1-\frac{\lambda_{6,2,1}+\lambda_{6,2,2}+\lambda_{6,2,3}}{85}} + \frac{\frac{\lambda_{6,5,1}+\lambda_{6,5,2}+\lambda_{6,5,3}}{90}}{1-\frac{\lambda_{6,5,1}+\lambda_{6,5,2}+\lambda_{6,5,3}}{90}} + \frac{\frac{\lambda_{6,7,1}+\lambda_{6,7,2}+\lambda_{6,7,3}}{60}}{1-\frac{\lambda_{6,7,1}+\lambda_{6,7,2}+\lambda_{6,7,3}}{60}}$$
$$+\frac{\frac{\lambda_{7,3,1}+\lambda_{7,3,2}+\lambda_{7,3,3}}{80}}{1-\frac{\lambda_{7,3,1}+\lambda_{7,3,2}+\lambda_{7,3,3}}{80}} + \frac{\frac{\lambda_{7,6,1}+\lambda_{7,6,2}+\lambda_{7,6,3}}{85}}{1-\frac{\lambda_{7,6,1}+\lambda_{7,6,2}+\lambda_{7,6,3}}{85}} + \frac{\frac{\lambda_{7,8,1}+\lambda_{7,8,2}+\lambda_{7,8,3}}{20}}{1-\frac{\lambda_{7,8,1}+\lambda_{7,8,2}+\lambda_{7,8,3}}{20}}$$
$$+\frac{\frac{\lambda_{8,4,1}+\lambda_{8,4,2}+\lambda_{8,4,3}}{80}}{1-\frac{\lambda_{8,4,1}+\lambda_{8,4,2}+\lambda_{8,4,3}}{80}} + \frac{\frac{\lambda_{8,5,1}+\lambda_{8,5,2}+\lambda_{8,5,3}}{95}}{1-\frac{\lambda_{8,5,1}+\lambda_{8,5,2}+\lambda_{8,5,3}}{95}} + \frac{\frac{\lambda_{8,7,1}+\lambda_{8,7,2}+\lambda_{8,7,3}}{70}}{1-\frac{\lambda_{8,7,1}+\lambda_{8,7,2}+\lambda_{8,7,3}}{70}}.$$

Optimal sub-streams are values of $\boldsymbol{\lambda_{i,j,k}}$, at which the value of $\boldsymbol{N(\lambda_{i,j,k})}$, taking into account the above equations and inequalities of constraints, will be **minimal**, but non-negative. The values of $\boldsymbol{\lambda_{i,j,k}}$ must also be non-negative.

2.4 The matrixes of the optimal sub-streams

Based on the adjacency matrix, the matrices of the desired optimal sub-streams are compiled.

Matrix of sub-streams of stream 1 (from node 1 to node 7):

$$\begin{bmatrix} 0 & \lambda_{1,2,1} & 0 & \lambda_{1,4,1} & \lambda_{1,5,1} & 0 & 0 & 0 \\ \lambda_{2,1,1} & 0 & \lambda_{2,3,1} & 0 & 0 & \lambda_{2,6,1} & 0 & 0 \\ 0 & \lambda_{3,2,1} & 0 & \lambda_{3,4,1} & 0 & 0 & \lambda_{3,7,1} & 0 \\ \lambda_{4,1,1} & 0 & \lambda_{4,3,1} & 0 & 0 & 0 & 0 & \lambda_{4,8,1} \\ \lambda_{5,1,1} & 0 & 0 & 0 & 0 & \lambda_{5,6,1} & 0 & \lambda_{5,8,1} \\ 0 & \lambda_{6,2,1} & 0 & 0 & \lambda_{6,5,1} & 0 & \lambda_{6,7,1} & 0 \\ 0 & 0 & \lambda_{7,3,1} & 0 & 0 & \lambda_{7,6,1} & 0 & \lambda_{7,8,1} \\ 0 & 0 & 0 & \lambda_{8,4,1} & \lambda_{8,5,1} & 0 & \lambda_{8,7,1} & 0 \end{bmatrix}$$

Matrix of sub-streams of stream 2 (from node 3 to node 5):

$$\begin{bmatrix} 0 & \lambda_{1,2,2} & 0 & \lambda_{1,4,2} & \lambda_{1,5,2} & 0 & 0 & 0 \\ \lambda_{2,1,2} & 0 & \lambda_{2,3,2} & 0 & 0 & \lambda_{2,6,2} & 0 & 0 \\ 0 & \lambda_{3,2,2} & 0 & \lambda_{3,4,2} & 0 & 0 & \lambda_{3,7,2} & 0 \\ \lambda_{4,1,2} & 0 & \lambda_{4,3,2} & 0 & 0 & 0 & 0 & \lambda_{4,8,2} \\ \lambda_{5,1,2} & 0 & 0 & 0 & 0 & \lambda_{5,6,2} & 0 & \lambda_{5,8,2} \\ 0 & \lambda_{6,2,2} & 0 & 0 & \lambda_{6,5,2} & 0 & \lambda_{6,7,2} & 0 \\ 0 & 0 & \lambda_{7,3,2} & 0 & 0 & \lambda_{7,6,2} & 0 & \lambda_{7,8,2} \\ 0 & 0 & 0 & \lambda_{8,4,2} & \lambda_{8,5,2} & 0 & \lambda_{8,7,2} & 0 \end{bmatrix}$$

Matrix of sub-streams of stream 3 (from node 6 to node 4):

$$\begin{bmatrix} 0 & \lambda_{1,2,3} & 0 & \lambda_{1,4,3} & \lambda_{1,5,3} & 0 & 0 & 0 \\ \lambda_{2,1,3} & 0 & \lambda_{2,3,3} & 0 & 0 & \lambda_{2,6,3} & 0 & 0 \\ 0 & \lambda_{3,2,3} & 0 & \lambda_{3,4,3} & 0 & 0 & \lambda_{3,7,3} & 0 \\ \lambda_{4,1,3} & 0 & \lambda_{4,3,3} & 0 & 0 & 0 & 0 & \lambda_{4,8,3} \\ \lambda_{5,1,3} & 0 & 0 & 0 & 0 & \lambda_{5,6,3} & 0 & \lambda_{5,8,3} \\ 0 & \lambda_{6,2,3} & 0 & 0 & \lambda_{6,5,3} & 0 & \lambda_{6,7,3} & 0 \\ 0 & 0 & \lambda_{7,3,3} & 0 & 0 & \lambda_{7,6,3} & 0 & \lambda_{7,8,3} \\ 0 & 0 & 0 & \lambda_{8,4,3} & \lambda_{8,5,3} & 0 & \lambda_{8,7,3} & 0 \end{bmatrix}$$

Using the method of sequential quadratic programming (line 95 of the *Optiflows.m* script), the optimal values of $\lambda_{i,j,k}$ are calculated.

The calculated optimal values of $\lambda_{i,j,k}$ are substituted into matrices for sub-streams.

Matrix of calculated optimal sub-streams of stream 1 (from node 1 to node 7):

Node numbers	1	2	3	4	5	6	7	8
1	0	38,6682355	0	20,7256491	18,6061154	0	0	0
2	0	0	13,3930823	0	0	25,2751532	0	0
3	0	0	0	0	0	0	13,3930823	0
4	0	0	0	0	0	0	0	20,7256491
5	0	0	0	0	0	4,97722655	0	13,62888885
6	0	0	0	0	0	0	30,25237975	0
7	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	34,35453795	0

Matrix of calculated optimal sub-streams of stream 2 (from node 3 to node 5):

Node numbers	1	2	3	4	5	6	7	8
1	0	0	0	0	2,629292942	0	0	0
2	2,629292942	0	0	0	0	42,4127344	0	0
3	0	45,04202734	0	0	0	0	2,957972658	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	42,4127344	0	0	0
7	0	0	0	0	0	0	0	2,957972658
8	0	0	0	0	2,957972658	0	0	0

Matrix of calculated optimal sub-streams of stream 3 (from node 6 to node 4):

Node numbers	1	2	3	4	5	6	7	8
1	0	0	0	1,638287373	0	0	0	0
2	1,638287373	0	17,79786966	0	0	0	0	0
3	0	0	0	17,79786966	0	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	32,10020925
6	0	19,43615703	0	0	32,10020925	0	9,463633722	0
7	0	0	0	0	0	0	0	9,463633722
8	0	0	0	41,56384297	0	0	0	0

2.5 Optimal subflows visualization

Based on the matrices of the calculated optimal sub-streams, these sub-streams are visualized in the form of directed graphs.

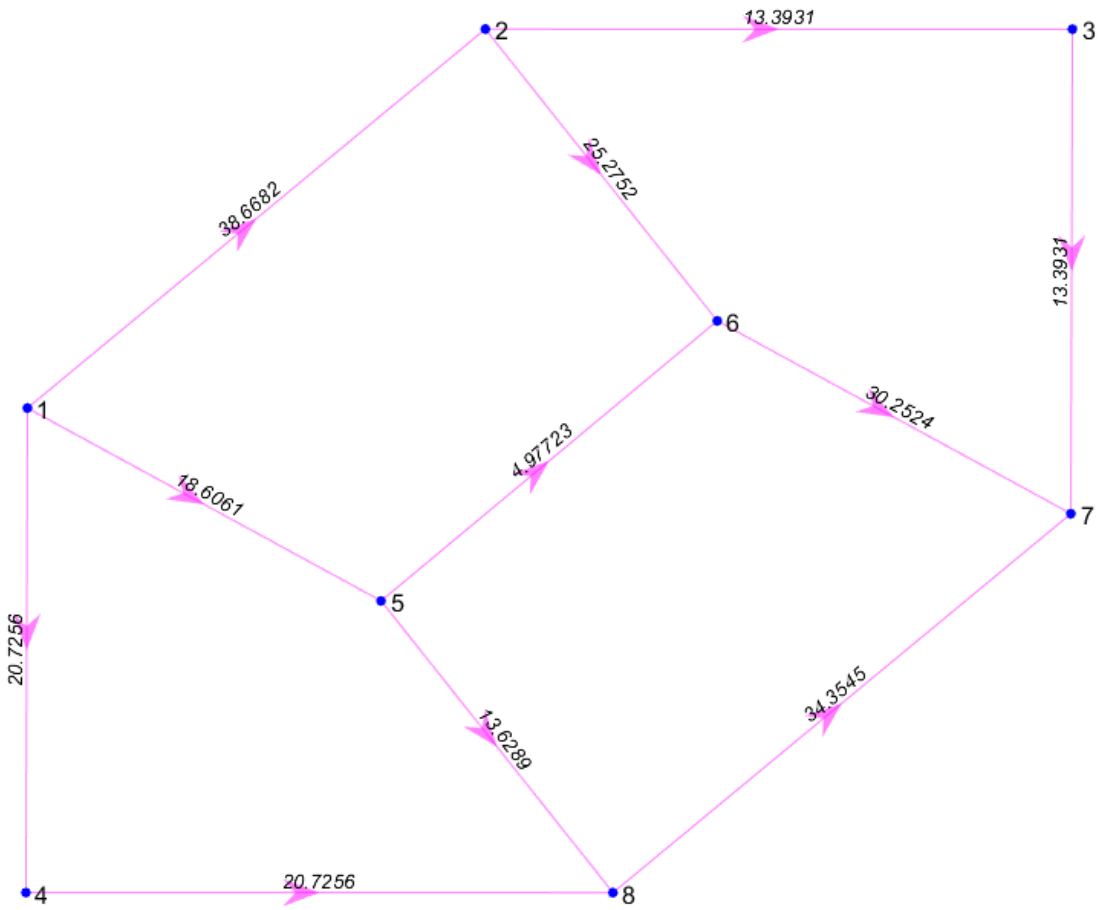

Figure 2.5.1 – directed graph of optimal sub-streams of stream 1 (from node 1 to node 7)

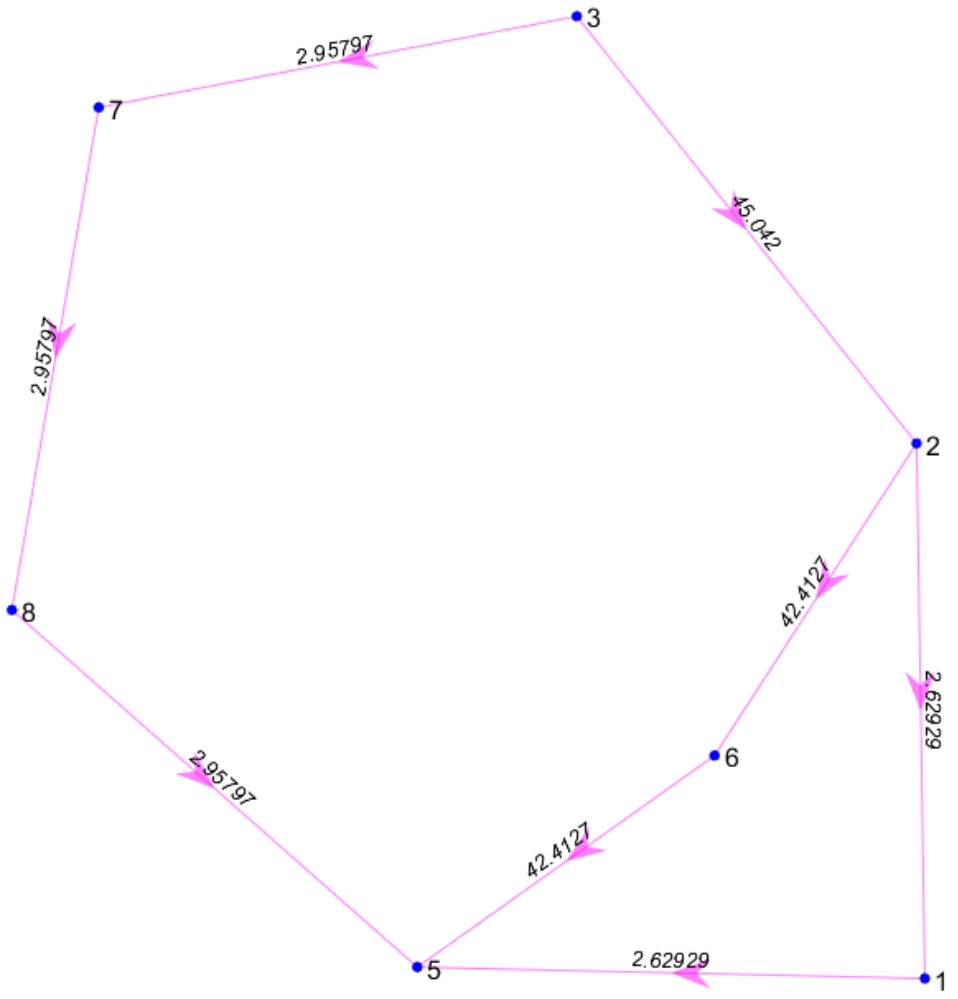

Figure 2.5.2 – directed graph of optimal sub-streams of stream 2 (from node 3 to node 5)

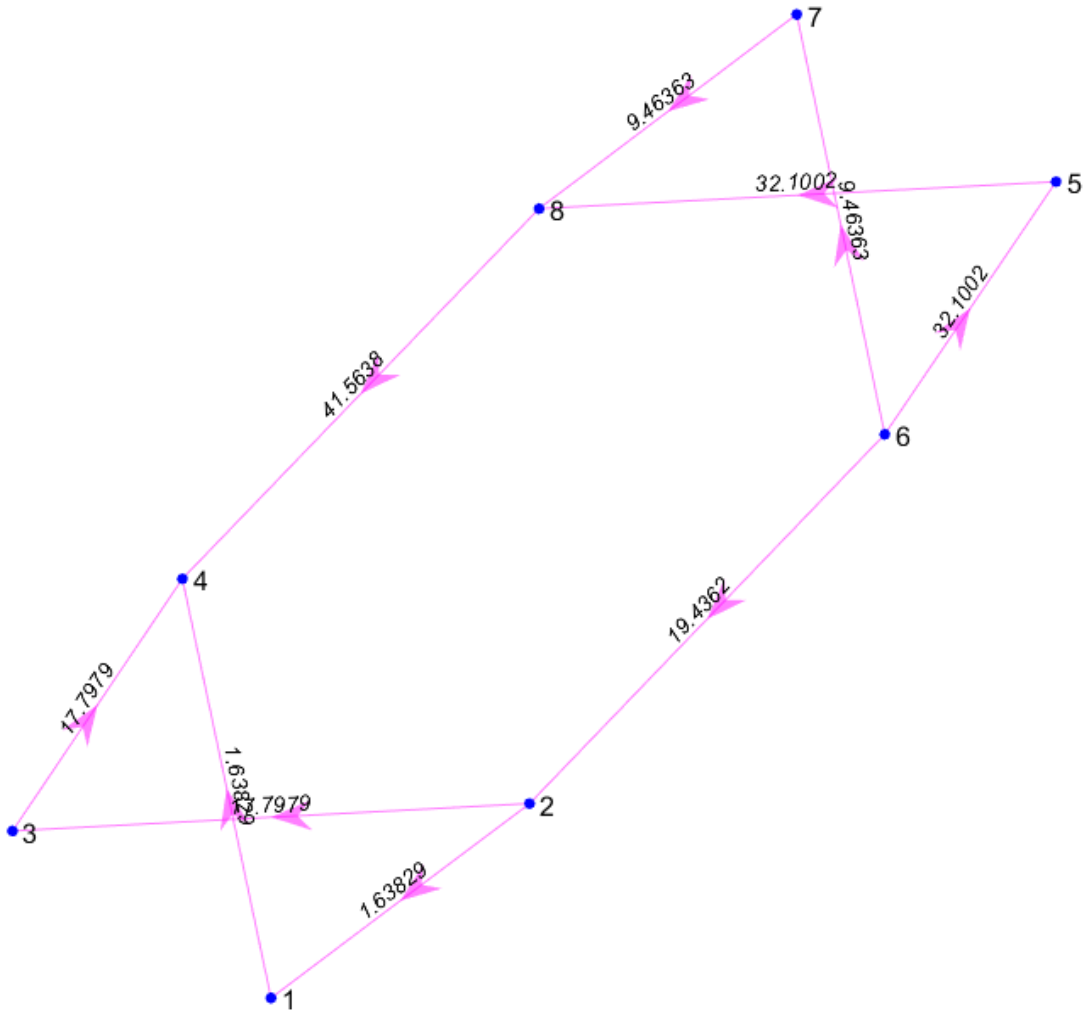

Figure 2.5.2 – directed graph of optimal sub-streams of stream 3 (from node 6 to node 4)

The presented directed graphs show node numbers, directions of optimal sub-streams, and their values. These images show that all the specified necessary conditions are met: the sum of outgoing sub-streams of the source node is equal to the sum of incoming sub-streams of the receiver node; the sum of incoming sub-streams to the intermediate node is equal to the sum of outgoing sub-streams from the intermediate node.

Thus, the optimization of streams in this network has been completed successfully.

3. Optimization algorithm program code

This section presents an optimization software package with line numbers, developed in the mathematical programming language MATLAB. The software package consists of an ***Optiflow.m*** script file containing the main algorithm code; and a ***TargetFunction.m*** function file containing the target function formation code.

For convenient visualization of information, the source data and calculation results are presented in the form of xls-tables.

NetMatrix.xlsx source file contains a table of the structure of the information network, a ***FlowMatrix.xlsx*** file contains a table of the stream direction structure in this network.

Based on the results of calculations ***OptiFlows.xlsx*** and ***LoadCoeffs.xlsx*** files are generated. The ***OptiFlows.xlsx*** file contains tables of optimal distribution of sub-streams, the LoadCoeffs.xlsx file contains the load coefficients of communication channels..

If optimization is not possible, the file ***OptiFlows.xlsx*** it is not created, but the file ***LoadCoeffs.xlsx*** is created at the same time, which contains information about how many times it is necessary to increase certain communication channels in order to perform successful optimization.

3.1 OptiFlows.m script

```
1. nM = readmatrix('NetMatrix.xlsx');
2. fM = readmatrix('FlowMatrix.xlsx');
3. qn = length(nM);
4. global Mu nFlows
5. nEdges = nnz(nM);
6. target = zeros( 1, nEdges );
7. source = target;
8. Mu = source;
9. u = 1;
10. for p = 1 : qn
11.     for q = 1 : qn
12.         if nM(p, q) > 0
13.                 source(u) = p;
14.                 target(u) = q;
15.                 Mu(u) = nM(p, q);
16.                 u = u + 1;
17.          end
18.       end
19. end
20. iFlows = zeros(qn, nEdges);
21. for l = 1 : nEdges
22.     iFlows(source(l), l) = -1;
23.     iFlows(target(l), l) =  1;
24. end
25. nFlows = nnz(fM);
26. ftarget = zeros(1, nFlows );
27. fsource = ftarget;
28. flows = fsource;
29. u = 1;
30. for p = 1 : qn
31.         for q = 1 : qn
32.             if fM(p, q) > 0
33.                     fsource(u) = p;
34.                     ftarget(u) = q;
35.                     flows(u) = fM(p, q);
36.                     u = u + 1;
37.             end
```

```
        end
end
fST = zeros(qn, nFlows);
for l=1 : nFlows
    fST(fsource(l), l) = -flows(l);
    fST(ftarget(l), l) =  flows(l);
end
flowSourceTarget = zeros(qn + 2, nFlows);
flowSourceTarget(1 : qn, :) = fST;
stLimit = zeros(2, nEdges, nFlows);
for v = 1 : nFlows
    for w = 1 : nEdges
        if source(w) == ftarget(v)
            stLimit(1, w, v) = 1;
        elseif target(w) == fsource(v)
            stLimit(2, w, v) = 1;
        end
    end
end
QFlows = zeros(qn + 2, nEdges, nFlows);
for o = 1 : nFlows
QFlows(1 : qn, :, o)=iFlows;
QFlows(qn + 1 : qn + 2, :, o) = stLimit(:, :, o);
end
for k = 1 : nFlows
    for i = 1 : qn
        switch sign(flowSourceTarget(i, k))
            case -1
                for j = 1 : nEdges
                    if QFlows(i, j, k) == 1
                        QFlows(i, j, k) = 0;
                    end
                end
            case 1
                for j = 1 : nEdges
                    if QFlows(i, j, k) == -1
                        QFlows(i, j, k) = 0;
                    end
                end
        end
    end
end
Aeq = zeros((qn + 2) * nFlows, nEdges * nFlows);
Beq = zeros(( qn + 2) * nFlows, 1);
for j = 0 : nFlows - 1
   Aeq((qn + 2) * j + 1 : (qn + 2) * (j + 1), nEdges * j + 1 : nEdges * (j + 1)) = QFlows(:, :, j + 1);
   Beq((qn + 2) * j + 1 : (qn + 2) * (j + 1), 1) = flowSourceTarget(:, j + 1);
end
Lambda0 = zeros(1, nFlows * nEdges);
A = zeros(nEdges, nEdges*nFlows );
for z=1:nEdges
    for y = 0 : nFlows - 1
       A(z, z + y * nEdges) = 1;
    end
end
B(:, 1) = Mu;
options = optimset('Algorithm', 'sqp', 'TolCon', 2^-32, 'MaxFunEvals', 2^18);
[Lambda, N] = fmincon(@TargetFunction, Lambda0, A, B, Aeq, Beq, Lambda0, [], [], options);
oFlows=zeros(nFlows,nEdges);
for n = 0 : nFlows - 1
    oFlows(n + 1, :) = Lambda(n * nEdges + 1 : nEdges * (n + 1));
end
for i = 1 : nFlows
    for j = 1 : nEdges
        if oFlows(i, j) < 10^-10
```

```
                        oFlows(i, j) = 0;
        end
    end
end
OFQM = zeros(qn, qn, nFlows);
for k = 1 : nFlows
   for x = 1 : nEdges
       OFQM(source(x), target(x), k) = oFlows(k, x);
   end
end
qL = sum(oFlows)./Mu;
qLM = zeros(qn);
for v = 1 : nEdges
     qLM(source(v), target(v)) = qL(v);
end
if N > 0
    disp('Optimization completed.');
    for i=1:nFlows
    writematrix(OFQM(:, :, i), 'OptiFlows.xlsx', 'Sheet', i);
    end
else disp("Optimization impossible. Need to increase the bandwidth of the channels.");
end
writematrix(qLM, 'LoadCoeffs.xlsx');
```

3.2 TargetFunction.m function

```
function MiddleQueue = TargetFunction(LambdaFlows)
global Mu nFlows
fS = zeros(1, length(Mu));
MiddleQueue = 0;
for i = 1 : length(Mu)
     for j = 0 : nFlows - 1
         fS(i) = fS(i) + LambdaFlows(length(Mu) * j + i);
     end
 end
for k = 1 : length(Mu)
     MiddleQueue = MiddleQueue + (fS(k) / Mu(k)) / (1-fS(k) / Mu(k));
end
end
```

4. Program code line-by-line comments

4.1 OptiFlows.m script

```
1. nM = readmatrix('NetMatrix.xlsx');
% Initialization of the nM matrix - reading the initial data of the network matrix
% structure.

2. fM = readmatrix('FlowMatrix.xlsx');
% Initialization of the nM matrix – reading the source data of the stream matrix structure.

3. qn = length(nM);
% Initialization of qn constant (number of nodes), equal to the length of the row of the
% nM matrix.

4. global Mu nFlows
% Declaring global variables Mu and nFlows for further use in the objective function.

5. nEdges = nnz(nM);
% Initializing nEdges constant (number of edges of the graph), equal to the number of
% nonzero elements in the nM matrix.

6. target = zeros( 1, nEdges );
7. source = target;
8. Mu = source;
% Initialization of target, source, Mu. Equating them to a null array of 1 row size,
% nEdges columns.

9. u = 1;
% Initializing u=1 index.

10. for p = 1 : qn
% The beginning of the cycle in lines 10 – 19 with a variable index p, varying from 1 to qn
% (the number of nodes).

11.    for q = 1 : qn
% The beginning of the cycle in lines 11 - 18 with a variable index q, varying from 1 to qn
% (the number of nodes).

12.        if nM(p, q) > 0
% The beginning of the condition in lines 12 – 17. If the element of the nM matrix under the
% indices p, q is greater than zero, then:

13.                 source(u) = p;
14.                 target(u) = q;
% Making of arrays-strings of numbers of source nodes and receiver nodes;
% source is array-string of numbers of source nodes, target is array-a string of numbers of
% receiver nodes.

15.                 Mu(u) = nM(p, q);
% Creating an array-string of bandwidth values.

16.                 u = u + 1;
% Index u increment.

17.         end
% The end of the condition in lines 12 – 17.

18.      end
%  The end of the loop in lines 11 – 18.

19. end
%  The end of the loop in lines 10 – 19.
```

```
20. iFlows = zeros(qn, nEdges);
% Initialization of the array of the information network incident matrix. The nFlows array
is equated to a zero matrix with the size of qn rows, nEdges columns.

21. for l = 1 : nEdges
% The beginning of the cycle in lines 21 – 24 with a variable index l, varying from 1 to
% nEdges (the number of communication channels).

22.     iFlows(source(l), l) = -1;
23.     iFlows(target(l), l) =  1;
% Creating a network incident matrix.

24. end
%  The end of the loop in lines 21 – 24.

25. nFlows = nnz(fM);
% Initialization of the nFlows constant (the number of streams in the network) equal to the
% number of non-zero elements in the fM matrix.

26. ftarget = zeros(1, nFlows );
27. fsource = ftarget;
28. flows = fsource;
% Initialization of variables target, source, flows. Equating them to a null array of 1 row,
% nFlows columns size.
29. u = 1;
% Initialization of u=1 index.

30. for p = 1 : qn
% The beginning of the cycle in lines 30 – 39 with a variable index p, varying from 1 to qn
% (the number of nodes).

31.        for q = 1 : qn
The beginning of the cycle in lines 31 – 38 with a variable index q, varying from 1 to qn
% (number of nodes).

32.            if fM(p, q) > 0
% The beginning of the condition in lines 32 – 37. If the element of the fM matrix under the
indices p, q is greater than zero, then he condition in these lines is met:

33.                    fsource(u) = p;
34.                    ftarget(u) = q;
% Creating arrays-strings of numbers of terminal source nodes and terminal receiver nodes;
% fsource is array-string of numbers of terminal source nodes, ftarget is array-string of
% numbers of terminal receiver nodes.

35.                    flows(u) = fM(p, q);
% Creating an array-string of stream values.

36.                    u = u + 1;
% u index increment.

37.            end
% The end of the condition in lines 32 – 37.

38.        end
%  The end of the loop in lines 31 – 38.

39. end
%  The end of the loop in lines 30 – 39

40. fST = zeros(qn, nFlows);
% Initialization of the array of the streams constraints matrix in the terminal communica-
tion channels. Equating them to a null matrix with qn rows, nFlows columns.
```

```
41. for l=1 : nFlows
% The beginning of the cycle in lines 41 – 44 with a variable index l, varying from 1 to
% nFlows (the number of sub-streams).

42.     fST(fsource(l), l) = -flows(l);
43.     fST(ftarget(l), l) =  flows(l);
% Creating a matrix of stream constraints in terminal communication channels.

44. end
%  The end of the loop in lines 41 – 44.

45. flowSourceTarget = zeros(qn + 2, nFlows);
% Initialization of flowSourceTarget array with dimension qn + 2 rows, nFlows columns.

46. flowSourceTarget(1 : qn, :) = fST;
% Assigning the first qn rows of the flowSourceTarget array to the value of the fST array.
% In the further part of the code, the flowSourceTarget array will be used as the right side
% of the constraint equations.

47. stLimit = zeros(2, nEdges, nFlows);
% Initialization of a three-dimensional stLimit array with a dimension of 2 rows, nEdges of
% columns, nFlows of matrices. In the further part of the code, this array will create the
% following optimization constraints: the sum of incoming sub-streams to source nodes is
% zero, the sum of outgoing sub-streams from receiver nodes is zero.

48. for v = 1 : nFlows
% The beginning of the cycle in lines 48 – 56 with a variable index v, varying from 1 to
% nEdges (the number of sub-streams).

49.     for w = 1 : nEdges
% The beginning of the cycle in lines 49 – 55 with a variable index w, varying from 1 to
% nEdges (the number of communication channels).

50.         if source(w) == ftarget(v)
% The beginning of the condition in lines 50 – 51. If the number of the source node
% source(w) matches the number of the terminal receiver node ftarget(v), then:
51.             stLimit(1, w, v) = 1;
% the value of the array elements stLimit(1, w, v) will be assigned to one.

52.         elseif target(w) == fsource(v)
% The beginning of the condition in lines 52 – 53. If the number of receiver node target(w)
coincides with the number of the terminal source node source(v), then:

53.             stLimit(2, w, v) = 1;
% the value of the stLimit(1, w, v) array element will be assigned one.

54.         end
% The end of the conditions in lines 50 – 54.

55.     end
%  The end of the loop in lines 49 – 55.

56. end
%  The end of the loop in lines 48 – 56.

57. QFlows = zeros(qn + 2, nEdges, nFlows);
% Initialization of a three-dimensional QFlows array with dimension qn + 2 rows,
% nEdges columns, nFlows matrices.

58. for o = 1 : nFlows
% The beginning of the cycle in lines 58 – 61 with a variable index o, varying from 1 to
% nFlows (the number of sub-streams).
```

```
59.    QFlows(1 : qn, :, o) = iFlows;
% Assigning the values of the iFlows incidence matrix to the elements of the QFlows array
% (from 1 to qn rows, all columns, matrix with index o).

60.    QFlows(qn + 1 : qn + 2, :, o) = stLimit(:, :, o);
% Assignment to the elements of the QFlows array of the matrix value with the index o of the
three-dimensional array stLimit (from qn+1 to qn+2 rows, all columns, matrix with index o).

61. end
%  The end of the loop in lines 58 – 61.

62. for k = 1 : nFlows
% The beginning of the cycle in lines 62 – 79 with a variable index k, varying from 1 to
% nFlows (the number of sub-streams). This loop adds the following optimization constraints:
% only the sum of the outgoing sub-streams of the source nodes is equal to the value of the
% outgoing stream, only the sum of the incoming sub-streams of the receiver nodes is equal
% to the value of the incoming stream.

63.     for i = 1 : qn
% The beginning of the cycle in lines 63-78 with a variable index i, varying from 1 to qn
% (the number of nodes).

64.         switch sign(flowSourceTarget(i, k))
% The beginning of the condition in lines 64-77 with the sign parameter (+ or -) of the
% element of the flowSourceTarget array.

65.             case -1
% If the element of the flowSourceTarget(i, k) array is negative, then lines 66 – 70
% are executed.

66.                 for j = 1 : nEdges
% The beginning of the cycle in lines 66 – 70 with a variable index j, varying from 1
% to nEdges (the number of communication channels).

67.                     if QFlows(i, j, k) == 1
% The beginning of the condition in lines 67 – 69. If an element of the QFlows array with
% indexes i, j, k equals one, then this condition will be met.

68.                         QFlows(i, j, k) = 0;
% The element of the QFlows array with indexes i, j, k is assigned zero.

69.                     end
% The end of the condition is in lines 67 – 69.

70.                 end
%  The end of the loop in lines 66 – 70.

71.             case 1
% If the element of the flowSourceTarget(i, k) array is positive,
% then lines 72 – 76 are executed.

72.                 for j = 1 : nEdges
% The beginning of the cycle in lines 72 – 76 with a variable index j,
% varying from 1 to nEdges (the number of communication channels).

73.                     if QFlows(i, j, k) == -1
% The beginning of the condition in lines 73 – 75. If an element of the QFlows array with
% indexes i, j, k equals minus one, then line 74 is executed.

74.                         QFlows(i, j, k) = 0;
% The element of the QFlows array with indexes i, j, k is equal to zero.

75.                     end
% The end of the condition in the lines 73 – 75.
```

```
76.                 end
%  The end of the loop in lines 72 – 76.

77.         end
% The end of the condition in the lines 64 – 77.

78.     end
%  The end of the loop in lines 63 – 78.

79. end
%  The end of the loop in lines 62 – 79.

80. Aeq = zeros((qn + 2) * nFlows, nEdges * nFlows);
% Initialization of an Aeq array with dimension (qn+2)*nFlows of rows, nEdges*nFlows of
% columns. In the following part of the code, the Aeq array will be used as the left side of
% the constraint equations of the objective function.

81. Beq = zeros(( qn + 2) * nFlows, 1);
% Initialization of the Beq array with dimension (qn+2)*nFlows rows, one column.
% In the following part of the code, the Beq array will be used as the right side of the
% constraint equations of the objective function.

82. for j = 0 : nFlows – 1
% The beginning of the cycle in lines 82 – 85 with a variable index j, varying from 1 to
% nFlows-1 (the number of sub-streams minus one). This loop creates arrays of linear
% constraints of the objective function.

83.     Aeq((qn + 2) * j + 1 : (qn + 2) * (j + 1), nEdges * j + 1 : nEdges * (j + 1)) = QFlows(:, :, j + 1);
% Creating the left side of the constraint equations of the objective function.

84.     Beq((qn + 2) * j + 1 : (qn + 2) * (j + 1), 1) = flowSourceTarget(:, j + 1);
% Creating the right side of the constraint equations of the objective function.

85. end
%  The end of the loop in lines 82 – 85.

86. Lambda0 = zeros(1, nFlows * nEdges);
% Initialization of the array of optimal sub-streams being searched for. The dimension of
% the array is 1 row, nFlows*nEdges columns.

87. A = zeros(nEdges, nEdges * nFlows );
% Initialization of the array of the left part of the inequalities of the constraints of the
% values of the sub-streams. The dimension of the array is nEdges rows,
% nEdges*nFlows columns.

88. for z = 1 : nEdges
% The beginning of the cycle in lines 88 – 92 with a variable z index varying from 1 to
% nEdges (number of communication channels). In this cycle, the values of array A are
% created, which is the left part of the inequalities of the constraints of the searchable
% optimal sub-streams.

89.     for y = 0 : nFlows – 1
% The beginning of the cycle in lines 89 – 91 with a variable index y, varying from 1 to
% nFlows-1 (the number of sub-streams minus one).

90.         A(z, z + y * nEdges) = 1;
% The elements of array A with indexes z, z+y*nEdges are equated to one.

91.     end
%  The end of the loop in lines 89 – 91.

92. end
%  The end of the loop in lines 88 – 92.
```

```
93. B(:, 1) = Mu;
% Initialization and creating of an array-column B, equal to the values of the maximum
% bandwidth of communication channels. Array B is the right part of the inequality
% of constraints of searchable optimal sub-streams.

94. options = optimset('Algorithm', 'sqp', 'TolCon', 2^-32, 'MaxFunEvals', 2^18);
% Setting parameters of the optimization calculation method. In this parameter, a sequential
% quadratic programming algorithm is selected, the accuracy of calculations is specified,
% and the maximum number of iterations is specified.

95. [Lambda, N] = fmincon(@TargetFunction, Lambda0, A, B, Aeq, Beq, Lambda0, [], [], options);
% Accessing the fmincon function to calculate the optimal values of sub-streams. The input arguments of
% the fmincon function are: objective function, initialization of required values in the form of an
% array Lambda0, constraint inequalities A<=B, constraint equations Aeq=Beq, the minimum required
% values in the form of an array Lambda0, parameters of the calculation method.


96. oFlows=zeros(nFlows,nEdges);
% Initialization of the oFlows array with the dimension of nFlows rows, nEdges columns.

97. for n = 0 : nFlows - 1
% The beginning of the cycle in lines 97-99 with a variable index n, varying from 1 to
% nFlows-1 (the number of streams minus one).

98.     oFlows(n + 1, :) = Lambda(n * nEdges + 1 : nEdges * (n + 1));
% Converting a one-dimensional Lambda string array into a two-dimensional oFlows
% matrix array.

99. end
%  The end of the loop in lines 97 – 99.

100. for i = 1 : nFlows
% The beginning of the cycle in lines 100 – 106 with a variable index i, varying from 1 to
% nFlows (the number of sub-streams).

101.    for j = 1 : nEdges
% The beginning of the cycle in lines 101 – 105 with a variable index j, varying from 1 to
% nEdges (the number of communication channels).

102.         if oFlows(i, j) < 10^-10
% The beginning of the condition in lines 102 – 104.

103.                         oFlows(i, j) = 0;
% Due to a possible insignificant optimization inaccuracy, with a negligibly small value of
% the oFlows(i, j) sub-stream, this value is assigned zero. As a rule, the inaccuracy does
% not exceed the order of 10^-17.

104.         end
% The end of the condition in the lines 102 – 104.

105.     end
%  The end of the loop in lines 101 – 105.

106. end
%  The end of the loop in lines 100 – 106.

107. OFQM = zeros(qn, qn, nFlows);
% Initialization of a three-dimensional OFDM array with the dimension of qn rows,
% qn columns, nFlows matrices.

108. for k = 1 : nFlows
% The beginning of the cycle in lines 108 – 112 with a variable index k, varying from 1 to
% nFlows (the number of streams).
```

```
109.    for x = 1 : nEdges
% The beginning of the cycle in lines 109 – 111 with a variable index x, varying from 1 to
% nEdges (the number of communication channels).

110.        OFQM(source(x), target(x), k) = oFlows(k, x);
% Converting of a two-dimensional oFlows array into a three-dimensional array OFQM.
% Thus, the result of software calculations is a three-dimensional OFDM array.

111.    end
%  The end of the loop in lines 109 – 111

112. end
%  The end of the loop in lines 108 – 112.

113. qL = sum(oFlows)  ./ Mu;
% Creating a qL array, which is the ratio of the sums of sub-streams in each communication
node to the maximum bandwidth of each communication channel.

114. qLM = zeros(qn);
% Initialization of the qLM array as a square matrix of size qn*qn.

115. for v = 1 : nEdges
% The beginning of the cycle in lines 115 – 117 with a variable index v, varying from 1 to
nEdges (the number of communication channels).

116.      qLM(source(v), target(v)) = qL(v);
% Converting a qL string to a qLM matrix. The qLM array is a matrix of loading of
% communication channels, which displays the ratio of the sums of sub-streams in each
% communication channel to the maximum bandwidth of each communication channel. If at least
% one element of the array exceeds one, then optimization is not possible with the current
% source data, and for successful optimization, it is necessary to increase the bandwidth of
% those communication channels whose loading exceeds one in the qLM matrix by at least the
% number of times that is equal to the value of the elements of the qLM matrix
% exceeding one.

117. end
% The end of the loop in lines 115 – 117.

118. if N > 0
% The beginning of the condition in lines 118 – 124. If the calculated value of the average
% queue N is non-negative, then the actions in lines 119 – 122 are executing.

119.     disp('Optimization has been completed.');
% Outputting a message to the console that optimization has been completed.

120.     for i = 1 : nFlows
% The beginning of the cycle in lines 120-122 with a variable index i, varying from 1 to
% nFlows (the number of sub-streams).

121.     writematrix(OFQM(:, :, i), 'OptiFlows.xlsx', 'Sheet', i);
% Saving calculation results to a file OptiFlows.xlsx.

122.     end
% The end of the loop in lines 120 – 122.

123. else disp("Optimization impossible. Need to increase the bandwidth of the channels.");
% If the value of the average queue is negative, then a message is output to the console
% stating that optimization is impossible and it is necessary to increase the bandwidth of
% communication channels.

124. end
% The end of the loop in lines 118 – 124.

125. writematrix(qLM, 'LoadCoeffs.xlsx');
% Saving the values of loading communication channels to a file LoadCoeffs.xlsx.
```

4.2 TargetFunction.m function

```
1. function MiddleQueue = TargetFunction(LambdaFlows)
% Initialization of the TargetFunction function with the input arguments LambdaFlows
% (an array of optimized sub-streams), and the output argument MiddleQueue (scalar
% value of the value of the average queue of the package being in the service system).

2. global Mu nFlows
% Declaration of global variables Mu, nFlows.

3. fS = zeros(1, length(Mu));
% Initialization of an fS array with a dimension of one row and a number of columns equal
% to the number of communication channels.

4. MiddleQueue = 0;
% Equating the initial value of the middle queue to zero.

5. for i = 1 : length(Mu)
% The beginning of the cycle in lines 5 – 9 with a variable index i, varying from 1 to the
% value of the number of communication channels.

6.      for j = 0 : nFlows – 1
% The beginning of the cycle in lines 6 – 8 with a variable index j, varying from 1 to the
% value of the number of sub-streams minus one.

7.          fS(i) = fS(i) + LambdaFlows(length(Mu) * j + i);
% Summation of the values of the searchable optimal sub-streams passing through each i-th
communication channel.

8.      end
%  The end of the loop in lines 6 – 8.

9.  end
%  The end of the loop in lines 5 – 9.

10. for k = 1 : length(Mu)
% The beginning of the cycle in lines 10 – 12 with a variable index k, varying from 1 to the
% value of the number of communication channels.

11.     MiddleQueue = MiddleQueue + (fS(k) / Mu(k)) / (1-fS(k) / Mu(k));
% Creating an objective function.

12. end
%  The end of the loop in lines 10 – 12.

13. end
% The end of the TargetFunction function.
```

Epilogue

The developed software package can be implemented into an existing information network using Multiprotocol label switching Traffic Engineering technology, creating metrics and tunnels for each optimal stream based on the calculations performed.

The implementation of this software package can save up to 3.87% of the company's turnover, since according to the research organization Gartner, such a share of the enterprise is spent on the maintenance of the infocommunication infrastructure.

In this way, with the successful implementation of this software algorithm, the economic efficiency and productivity of enterprises will be significantly increased.